



Virtual Memory & I/O Systems

Brief Overview

This note covers **Virtual Memory & I/O Systems** and was created from a PDF of 193 pages. It gathers a comprehensive look at page size trade-offs, TLB behavior, RAID configurations, file system structures, and network I/O.

Key Points

- Understand how **page size** affects I/O efficiency and locality.
- Learn TLB concepts and their impact on address translation performance.
- Explore **RAID levels** and snapshot mechanisms for data protection.
- Review file system allocation, caching, and network socket interfaces.

Page Size Trade-offs

I/O Time vs. Page Size

- Larger pages reduce the **percentage of I/O time** spent on page-fault handling because fewer pages need to be transferred.
- Smaller pages improve **locality resolution**: only the memory actually used is brought in, reducing total I/O and allocated memory.
- Trade-off:
 - Very small pages (e.g., 1 byte) cause a page fault for each accessed byte, incurring high fault-handling overhead.
 - Very large pages may transfer unnecessary data, wasting bandwidth and memory.

Definition: *Locality* – the tendency of a program to access a relatively small set of pages repeatedly over a short period.

Example: 200 KB Process

Page Size	Pages Needed	Potential I/O
200 KB (single page)	1	200 KB transferred (even if only 100 KB used)
1 byte	200 000 pages	100 KB transferred (only used bytes) but 200 000 faults

TLB Reach and Hit Ratio

Definition: *TLB hit ratio* – the fraction of virtual-address translations satisfied by the Translation Lookaside Buffer (TLB) without consulting the page table.

Definition: *TLB reach* – the total amount of memory that can be addressed by the entries currently stored in the TLB.

$$\text{TLB reach} = (\text{number of TLB entries}) \times (\text{page size})$$

Effects of Page Size on TLB Reach

Page Size	TLB Entries (example)	TLB Reach
8 KB	64	512 KB
32 KB	64	2 MB
4 MB	64	256 MB

- Increasing page size **quadratically** expands reach (e.g., 8 KB $\rightarrow$ 32 KB quadruples reach).
- Larger reach helps keep a process's *working set* within the TLB, reducing page-table walks.

Inverted Page Tables

Definition: *Inverted page table* – a global table with one entry per **physical** page, indexed by . It reduces memory needed for page-table storage.

- Stores only the mapping from physical frames to the owning virtual page.
- Lacks full per-process logical address information; an **external page table** (traditional per-process table) is kept for demand paging.
- External tables are **paged in/out** only on page faults, potentially causing a *secondary* page fault while handling the first.

Program Structure and Locality

Row-major vs. Column-major Access

```
/* Row-major version (high page-fault count) */
int i, j;
int data[128][128];
for (j = 0; j < 128; j++)
    for (i = 0; i < 128; i++)
        data[i][j] = 0;

/* Column-major version (lower page-fault count) */
int i, j;
int data[128][128];
for (i = 0; i < 128; i++)
    for (j = 0; j < 128; j++)
        data[i][j] = 0;
```

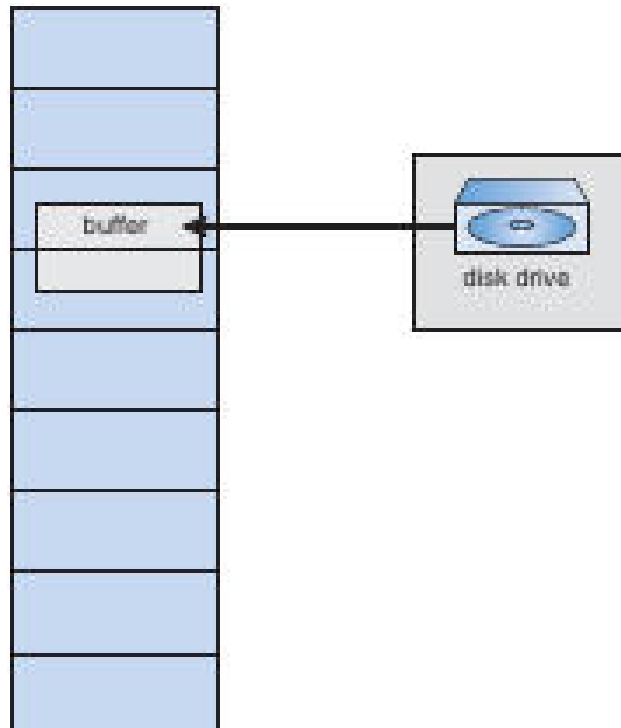
- In **row-major** layout, each row occupies one page; the outer loop over columns touches a new page each iteration $\rightarrow$ many faults.
- In **column-major** layout, the inner loop stays within the same row (page) longer, improving locality and reducing faults.

Definition: *Working set* – the set of pages a process actively uses during a time interval.

- Data structures with good locality (e.g., stacks) generate fewer faults; structures designed for scattering (e.g., hash tables) generate more.

I/O Interlock and Page Locking 🔒

The following diagram illustrates a typical buffer-to-disk relationship and why pages may need to be locked during I/O:



The image shows memory divided into six divisions, a buffer linked to a specific division, and the buffer's connection to a disk drive. It highlights that the buffer's physical frame must remain resident while the I/O controller accesses it.

- **Problem:** If the buffer page is swapped out before the I/O completes, the device may write to an incorrect frame.
- **Solutions:**
 1. **Copy-in/copy-out** – I/O operates on a kernel buffer; user pages are copied before/after transfer (adds overhead).
 2. **Page locking** – a *lock bit* on a frame prevents it from being selected for replacement during the I/O operation.

Lock-bit Management

- Set when a page is involved in an I/O request; cleared after completion.
- Overuse can lead to memory starvation; operating systems typically require privileged access to lock pages.

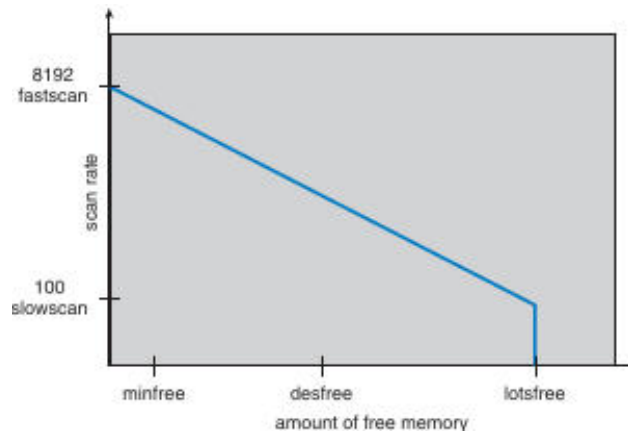
OS Implementations 📖

Windows Virtual Memory

- Uses **demand paging with clustering**: on a fault, the faulting page *and* several following pages are fetched.
- **Working-set model**: each process has a *minimum* and *maximum* number of resident pages.
- **Free-page list** and a **threshold (lotsfree)** determine when the system is “low on memory.”
- **Page replacement**: local LRU when a process exceeds its maximum; automatic working-set trimming restores free memory.

Solaris Virtual Memory

- Parameter **lotsfree** ($\approx$ total physical memory) triggers the **page-out daemon** when free pages drop below this level.
- **Page-out algorithm** employs two “hands” (clock algorithm) with a configurable **scanrate**:
 - **slow scan** $\approx$ 100 pages/s
 - **fast scan** $\approx$ (total pages)/2 per second
- **Handspread** controls the distance between the two hands; larger spread means longer time between reference-bit checks.
- **Priority paging** distinguishes pages belonging to shared libraries or regular files, allowing smarter reclamation.



The graph shows scan rate decreasing (from fastscan to slowscan) as the amount of free memory increases, illustrating how Solaris adapts its paging aggressiveness.

Summary of Key Virtual-Memory Concepts 📖

Concept	Key Point
Page size	Balances I/O efficiency vs. locality; optimal size depends on workload.
TLB reach	Larger pages increase reach; more entries improve hit ratio but cost more hardware.
Inverted page table	Saves memory by indexing physical frames; requires auxiliary per-process tables.
Locality & program structure	Row-major vs. column-major access patterns affect page-fault rates.

I/O interlock	Locked pages prevent corruption during asynchronous I/O.
OS policies	Windows: clustering + working-set limits; Solaris: dynamic scanrate & priority paging.

These notes capture the essential trade-offs, mechanisms, and OS strategies related to virtual memory, page sizing, TLB behavior, and paging policies.

Demand-Paged System Utilization & Multiprogramming

Case	CPU Utilization	Disk (paging) Utilization
a	13 %	97 %
b	87 %	3 %
c	13 %	3 %

- **When CPU usage is low and disk usage is high (case a)** the system is I/O-bound.
 - Adding more processes (increasing the degree of multiprogramming) can raise CPU utilization because the CPU will have work to do while other processes wait for I/O.
 - Paging is *helpful* here: the high disk activity shows that many pages are being faulted in/out; the system is already exploiting virtual memory.
- **When CPU usage is high and disk usage is low (case b)** the system is CPU-bound.
 - Raising multiprogramming would likely **decrease** overall throughput because the CPU is already saturated; additional processes would compete for CPU cycles without gaining I/O overlap.
 - Paging contributes little; most accesses hit in memory.
- **When both utilizations are low (case c)** the system is under-utilized.
 - Increasing multiprogramming can improve both CPU and disk usage, provided enough memory is available to avoid excessive page faults.
 - Paging is not presently helping; a larger working set may be needed.

Key takeaway: Adjusting the degree of multiprogramming is beneficial primarily when one resource (CPU or disk) is under-utilized while the other is saturated.

Base/Limit Registers vs. Page Tables

Definition: *Base/limit registers* store a single contiguous region of legal addresses for a process; any reference outside [base, base + limit) causes a trap.

- Using only base/limit registers cannot represent **non-contiguous** virtual-to-physical mappings required by paging.
- A page table provides a per-page translation, allowing a process's address space to be scattered across physical memory.
- Therefore, a system that relies solely on base/limit registers **cannot** simulate a full page-table mechanism.

TLB Miss / Page-Fault Scenarios 🔍

Scenario	What Happens
TLB miss, no page fault	The hardware walks the page table (or uses a software handler) to obtain the frame number, updates the TLB, and the instruction proceeds.
TLB miss + page fault	The page-table walk discovers that the needed page is not resident. The OS brings the page from disk, updates the page table, fills a frame, then updates the TLB before restarting the instruction.
TLB hit, no page fault	Translation is found instantly in the TLB; the memory reference proceeds without additional overhead.
TLB hit + page fault	Impossible under a correct implementation: a hit guarantees that the page is resident. If the page were not resident, the entry would not be in the TLB.

Thread State Changes on Memory Events 🧵

Figure 8.31 (thread states: **Running** → **Ready** → **Blocked**)

- **Running → Page fault** → The thread is moved to **Blocked** while the OS fetches the required page.
- **Running → TLB miss (page-table hit)** → No state change; the hardware or OS resolves the miss transparently.
- **Running → Address reference that triggers a page fault** → Same as the first case: the thread becomes **Blocked**.

Page-Fault Rate & Working-Set Evolution 📊

1. **Process start (pure demand paging)** – The page-fault rate is initially **high** because no pages are resident.
2. **Working set fully loaded** – Once the set of pages the process actively uses is in memory, the fault rate drops sharply; the process now runs mainly with **TLB hits** and memory accesses.
3. **Working set larger than available free memory** – Options for the OS designer:
 - **Increase physical memory** (adds frames).
 - **Reduce the working set** by swapping out less-used pages (e.g., using a more aggressive replacement algorithm).
 - **Employ pre-fetching / prepaging** to anticipate future accesses.
 - **Adjust the degree of multiprogramming** to free frames for the heavy process.

Copy-On-Write (COW) 📋

Definition: *Copy-on-write* is a technique where multiple processes share a single physical copy of a page marked read-only; on the first write attempt, the OS copies the page so the writer gets a private copy.

- **Beneficial circumstances:**
 - Forking a process (the child initially shares the parent's address space).
 - Implementing shared libraries or memory-mapped files where most accesses are reads.

- **Hardware support required:**
 - A **read-only** bit in the page-table entry (or a protection bit).
 - The ability to trap a **write-fault** so the OS can allocate a new frame and copy the contents.
-

Address Translation Example

- **Virtual address:** 0x11123456 (hex)
- **Page size:** 4 KB = 2^{12} bytes $\rightarrow$ offset = lower 12 bits.
- **Virtual page number (VPN)** = upper bits = 0x11123.

Translation steps

1. **Hardware** extracts VPN and offset.
2. **TLB lookup** – if miss, OS walks the page table (software).
3. **Page-table entry** provides the **frame number** (e.g., 0x5A3).
4. **Physical address** = (frame number $\ll$ 12) | offset.

Software handles page-faults (reading from backing store) and updates the page table; *hardware* performs the bit-shifts and combines the result.

Page-Fault Service Time & Acceptable Fault Rate

- **Service times:**
 - Clean page (not modified): 8 ms
 - Dirty page (modified): 20 ms
 - 70 % of replaced pages are dirty.
- **Effective service time per fault:**

$$0.3 \times 8 \text{ ms} + 0.7 \times 20 \text{ ms} = 2.4 \text{ ms} + 14 \text{ ms} = 16.4 \text{ ms}$$
- **Memory-access time:** $100 \text{ ns} = 1 \times 10^{-7} \text{ s}$.
- Desired average access $\leq 200 \text{ ns} = 2 \times 10^{-7} \text{ s}$.

Let p = page-fault rate (faults per memory reference).

Average access time = $(1-p) \times 100 \text{ ns} + p \times 16.4 \text{ ms}$

Set $\leq 200 \text{ ns}$:

$$(1-p) \times 10^{-7} + p \times 1.64 \times 10^{-2} \leq 2 \times 10^{-7}$$

Solving gives $p \leq 1.22 \times 10^{-5}$ ($\approx 0.0012\%$).

Effect of One Thread's Page Fault on Other Threads

- All user threads of a process share the **same address space**.
 - When one thread faults on a page (e.g., its stack), the OS must bring that page into memory before *any* thread can safely execute code that might reference it.
 - Consequently, **all threads of the same process are blocked** until the faulting page is resident.
-

Page-Replacement Algorithms for a 3-Frame Reference String

Reference string:

0723125346771054623

Algorithm	Page Faults
FIFO	12
LRU	11
Optimal	9

(Counts derived by simulating each algorithm with three frames.)

Page Table Conversion & Reference Bit Updates

- **System parameters:** 16-bit virtual addresses, 4 KB pages → 4-bit offset, 12-bit VPN.
- **Given page-table entries** (page → frame): see Figure 8.32 (not reproduced).

Conversions:

Virtual address (hex)	VPN	Offset	Frame (from table)	Physical address (hex)
0xE12C	0xE1	0x2C	frame 9	0x92C
0x3A9D	0x3A	0x9D	frame 13	0xD9D
0xA9D9	0xA9	0xD9	frame 15	0xFD9
0x7001	0x70	0x01	frame 0	0x001
0xACA1	0xAC	0xA1	frame 5	0x5A1

- For each successful translation, set the **reference bit** of the corresponding page-table entry to **1**.
- Example of a **page fault** address: 0xFFFF (VPN=0xFF) which is not present in the table.

Clock (Second-Chance) Algorithm Behavior

- **Fast pointer movement** → many pages have their reference bit cleared; the algorithm is aggressively evicting pages, indicating **high memory pressure**.
- **Slow pointer movement** → most pages retain a set reference bit; the system has **ample free frames**, and the clock sweeps slowly.

LFU vs. LRU & MFU vs. LRU Scenarios

- **LFU beats LRU** when the workload repeatedly accesses a small set of “hot” pages while others are rarely used (e.g., database index pages).
- **LRU beats LFU** when the access pattern exhibits **temporal locality** but frequencies change over time (e.g., looping over a sliding window).

- **MFU beats LRU** in scenarios where the most frequently used pages become *obsolete* after a burst (e.g., a large initial load of data that is later discarded).
- **LRU beats MFU** when recent accesses are a better predictor of future use than cumulative frequency (typical interactive workloads).

VAX/VMS FIFO + Free-Frame Pool

- Page fault, page not in pool** → The system selects a victim from the FIFO queue, writes it back if dirty, and frees a frame for the new page.
- Page fault, page already in pool** → The page is moved to the *resident set*; no additional free frame is needed because the pool already holds the page.
- Free-frame pool size = 1** → The algorithm **degenerates to pure FIFO** (no LRU buffer).
- Free-frame pool size = 0** → The system behaves like a **strict FIFO replacement** without any secondary buffering.

Multiprogramming Utilization Adjustments

Option	Expected Effect on Utilization (given 20% CPU, 97.7% disk)
a. Faster CPU	Little impact – CPU is not the bottleneck.
b. Bigger paging disk	Negligible – disk already saturated.
c. Increase degree of multiprogramming	Improves – more processes keep CPU busy while disk works.
d. Decrease degree of multiprogramming	Worsens – less overlap, lower CPU use.
e. More main memory	Improves – reduces page-fault rate, frees disk I/O.
f. Faster disk / multiple disks	Improves – lowers paging I/O time, allowing higher CPU usage.
g. Add prepaging	Improves – anticipates needed pages, cutting fault count.
h. Increase page size	Mixed – larger pages reduce fault frequency but increase I/O per fault; may help if locality is high.

Indirect Memory Load & Per-Process Frame Allocation

- An **indirect one-level fault** occurs when the instruction references a page that is not resident; the CPU first fetches the page containing the pointer, then the target page.
- With **per-process frame allocation**, each process receives a fixed quota of frames. When a fault occurs, the OS may need to **reclaim a frame from the same process** (if its quota is exhausted) before satisfying the request, potentially causing *secondary* faults.

Second-Chance (Clock) Replacement Policy

- **Gain vs. LRU:**
 - **Gain** – Simpler hardware implementation (single circular hand + reference bit).
 - **Loss** – May retain a page with a recent reference but low future utility (less precise than true LRU).
 - The algorithm gives each page a *second chance* by clearing its reference bit on the first sweep; only pages with a cleared bit are evicted.
-

Counter-Based Replacement Algorithm

Algorithm definition

1. **Initial counters:** all set to 0.
2. **On page reference:** increment the counter of the referenced frame.
3. **Periodically** (or on replacement) **decrement** counters of all frames (e.g., each timer tick).
4. **Selection:** evict the frame with the **smallest counter** (ties broken FIFO).

Fault count for reference string (4-frame)

Reference string: 1, 2, 3, 4, 5, 3, 4, 1, 6, 7, 8, 7, 8, 9, 7, 8, 9, 5, 4, 5, 4, 2, 5

- Simulating the above policy yields **16 page faults**.

Optimal algorithm (4 frames) – minimum possible faults for the same string = **13**.

Effective Memory Access Time with TLB

- **Memory access:** 2 accesses (page-table + data) = $2 \times 1 \mu\text{s} = 2 \mu\text{s}$.
- **TLB hit rate** = 80% → access time = $1 \mu\text{s}$ (single access).
- **TLB miss** = 20% → need page-table lookup ($2 \mu\text{s}$) + possible extra 2% page-fault overhead (negligible here).

$$\text{EMAT} = 0.80 \times 1 \mu\text{s} + 0.20 \times 2 \mu\text{s} = 1.2 \mu\text{s}$$

(If the 2% of the remaining accesses cause page faults, add the fault service time accordingly.)

Thrashing: Cause, Detection, Mitigation

Definition: *Thrashing* occurs when a system spends most of its time servicing page faults rather than executing user code, typically because the combined working sets exceed available memory.

- **Detection** – Monitor **high page-fault rates** together with **low CPU utilization** (e.g., CPU < 20% while disk > 80%).
 - **Mitigation strategies:**
 - Reduce the degree of multiprogramming.
 - Increase physical memory.
 - Adjust the working-set window (make it larger to avoid premature eviction).
 - Employ *pre-paging* or more aggressive replacement algorithms.
-

Working-Set Model Parameter Effects

- **Small window size (Δ)** → Frequent updates; the system may treat transient pages as active, causing **excessive page faults** and possible thrashing.
- **Large window size** → Smoother estimate of the working set; reduces fault frequency but may retain rarely used pages, increasing memory pressure.
- **Very high Δ** (approaching total runtime) → The working set becomes the entire address space, effectively disabling the model and leading to **low responsiveness**.

Buddy System Allocation Example

Assume a **1024 KB** arena managed by the buddy allocator.

Request	Action
6 KB	Split 8 KB block → allocate 6 KB (remaining 2 KB kept as a free buddy).
250 bytes	Split a 512-byte buddy → allocate 250 B (remaining 262 B kept).
900 bytes	Allocate from a 1 KB buddy (after merging smaller buddies if needed).
Release 250 B	Free its buddy; if the adjacent buddy is also free, coalesce to a larger block.
Release 900 B	Free its block; attempt coalescing with neighboring free buddies.
Release 6 KB	Free the 6 KB block; merge with its 2 KB buddy (if free) and continue upward as possible.

The buddy system guarantees that all free blocks are powers of two and that merging occurs only when both buddies are free.

Slab Allocation & Multithreading Scalability

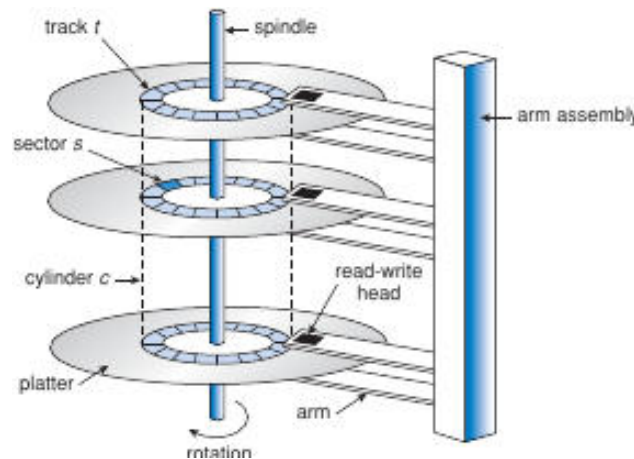
- **Slab allocator**: maintains a separate cache (slab) for each object size/type, reducing fragmentation and speeding up allocation/deallocation.
- **Scalability issue** with one-to-one user/kernel thread mapping: each user thread incurs a kernel-mode stack and control block, consuming kernel resources.
- **Solution**: use a **kernel-level thread pool** or **M:N threading model** where many user threads multiplex onto fewer kernel threads, decreasing the number of slabs needed for kernel objects.

Variable-Size Paging Advantages & Implementation

- **Advantages**:
 - Better fit for processes with heterogeneous locality (large code segment, small data segment). **Reduced internal fragmentation** compared with fixed-size pages.
 - Allows the OS to allocate **large pages** for hot regions (improving TLB reach) while keeping small pages for sparse regions.

- **System support:**
 - Hardware must provide **page-size bits** in the page-table entry to indicate the size of each mapping.
 - The OS maintains **segmentation-like structures** to track variable-size page boundaries and to perform address translation accordingly.

Hard-Disk Structure Overview ☒



The diagram shows a spindle holding multiple platters, each platter divided into tracks, sectors, and cylinders. A read-write head mounted on an arm assembly hovers just above the platter surfaces, illustrating the mechanical path of data reads and writes.

- **Spindle** rotates the platters, providing the baseline speed for sequential access.
- **Tracks** are concentric circles; a set of tracks with the same radius across all platters forms a **cylinder**.
- **Sectors** are the smallest addressable units on a track.
- The **arm assembly** moves the head to the correct cylinder; the head “flies” nanometers above the surface to read/write magnetic bits.

Understanding this structure is essential when evaluating **I/O interlock** and **page-locking** mechanisms discussed earlier.



Disk Physical Structure & Performance Characteristics

Definition: *Cylinders* – concentric circles on a platter; each cylinder contains one track per surface.

- A modern hard-disk platter can have **thousands of cylinders** and **hundreds of sectors** per track.
- **Capacity** is usually expressed in **gigabytes (GB)**.
- **Rotational speed** is given in **RPM** (revolutions per minute): common values are **5400**, **7200**, **10000**, and **15000 RPM**.
- **Transfer rate** = speed at which bits move between the disk surface and the computer.
- **Positioning time** = random-access time, composed of:
 1. **Seek time** – moving the actuator arm to the correct cylinder.
 2. **Rotational latency** – waiting for the desired sector to rotate under the head.
- Typical values: **transfer of several MB/s**, **seek times of a few ms**, **rotational latency of a few ms**.

Note: The read/write head “flies” on an air cushion only microns above the platter. Contact can cause a **head crash**, which usually renders the whole disk unrecoverable.

Removable Media

- **Removable hard disks** usually consist of a single platter protected by a plastic case.
 - Other removable storage includes **CDs, DVDs, Blu-ray discs**, and **flash-drive (SSD) devices**.
-



Solid-State Drives (SSDs)

Definition: *SSD* – a non-volatile memory device that uses flash cells (SLC, MLC, etc.) to store data like DRAM but without moving parts.

- **Advantages**
 - No moving parts → **zero seek time** and **very low latency**.
 - **Lower power consumption** and smaller physical footprint.
 - More reliable in harsh environments (no head-crash risk).
 - **Disadvantages**
 - Higher **cost per megabyte** than magnetic disks.
 - Limited write-endurance; lifespan can be shorter.
 - **Use cases**
 - Storing **filesystem metadata** for high-performance access.
 - Primary storage in laptops and performance-critical servers.
 - **Interface considerations**
 - Some SSDs connect directly to the system bus (e.g., PCIe) to avoid the bottleneck of SATA.
 - Because SSD access time is uniform, many OS disk-scheduling algorithms become unnecessary (see Section on Scheduling).
-



Magnetic Tape Storage

Definition: *Magnetic tape* – a sequential-access secondary-storage medium used for large-volume archival data.

- **Characteristics**
 - Very high **capacity** (several terabytes per cartridge).
 - **Slow random access** – moving the tape to the correct position can take minutes.
 - Once positioned, **data transfer rate** can approach that of hard disks.
 - **Typical formats** – widths of **4 mm, 8 mm, 12.7 mm**, and **1/4-inch**; technology names such as **LTO-5, SDLT**.
 - **Compression** – many tapes include built-in compression, often doubling effective storage.
 - **Role** – primarily for **backups** and **offline data transfer**, not for active secondary storage.
-



Logical Block Addressing & Disk Geometry

Definition: *Logical block* – the smallest addressable unit of storage on a disk, typically **512 bytes** (some disks support **1024 bytes**).

- Disks are presented to the OS as a **one-dimensional array of logical blocks**.

- **Mapping**: logical block0 corresponds to the first sector on the outermost track of the outermost cylinder; subsequent blocks follow the track, then move to the next track/cylinder inward.
- **Variable sectors per track**
 - **CAV (Constant Angular Velocity)**: spindle rotates at constant speed; outer tracks contain more sectors than inner tracks.
 - **CLV (Constant Linear Velocity)** (used in optical media): bit density is uniform across the radius.
- Modern drives include **spare sectors** to replace defective ones transparently.

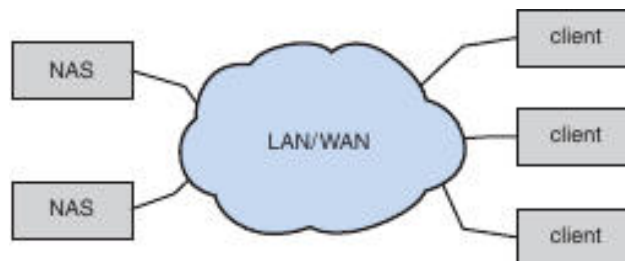
🔌 Disk Attachment Technologies

Host-Attached Storage (HAS)

Definition: *Host-attached storage* – disks directly connected to a computer via a local I/O bus.

- Common bus standards: **ATA/IDE**, **SATA**, **eSATA**, **USB**, **PCIe**, and **Fibre Channel (FC)**.
- **Disk controller** resides on the drive; a **host controller** on the host side manages commands and may include a **cache**.
- Data flow:
 1. Application → OS → **host controller** (via memory-mapped I/O).
 2. Host controller → **disk controller** (via bus).
 3. Disk controller ↔ **drive cache** ↔ **disk surface**.

Network-Attached Storage (NAS)



The diagram illustrates NAS devices and client computers sharing storage over a LAN/WAN.

Definition: *NAS* – a dedicated storage appliance that provides file-level access (e.g., NFS, CIFS) over a network.

- Implements **RPC** calls (NFS for UNIX, CIFS for Windows) over **TCP/UDP**.
- Typically built on a **RAID array** for reliability.
- Advantages: **centralized management**, **easy expansion**.
- Drawbacks: **higher latency** and **lower throughput** compared with direct-attached storage.

Storage-Area Network (SAN)

Definition: *SAN* – a high-speed, private network that provides block-level access to storage devices, usually via **Fibre Channel** or **iSCSI**.

- Connects **multiple servers** to **multiple storage arrays** through a **switch fabric**.
- Offers **dynamic allocation** of storage to hosts and supports clustering.
- **FC** is the dominant SAN interconnect; **InfiniBand** is an emerging alternative.

Disk Scheduling Algorithms

Goal: Minimize total **seek time** (and, on rotating media, **rotational latency**) by ordering pending I/O requests.

First-Come-First-Served (FCFS)

- Simple queue, service requests in arrival order.
- Often results in large head swings and poor performance.

Shortest-Seek-Time-First (SSTF)

- Selects the pending request **closest** to the current head position.
- Reduces total head movement but can cause **starvation** of distant requests.

SCAN (Elevator)

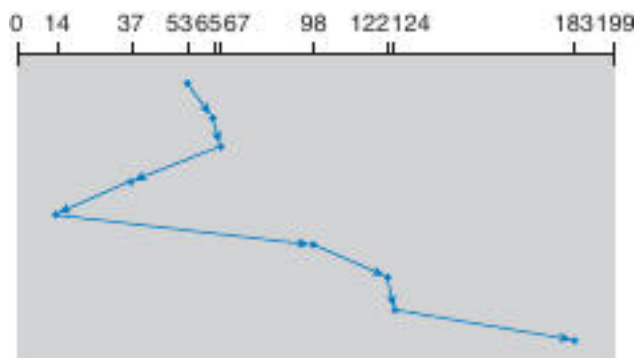
- Head moves **in one direction**, servicing all requests until it reaches the end, then reverses.
- Provides more uniform wait times than SSTF.

C-SCAN (Circular SCAN)

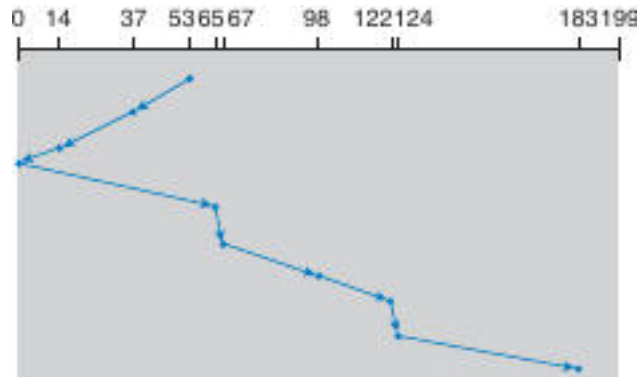
- Similar to SCAN, but after reaching the far end, the head **jumps back** to the beginning without servicing requests on the return.
- Yields **more predictable** wait times across the disk.

LOOK & C-LOOK

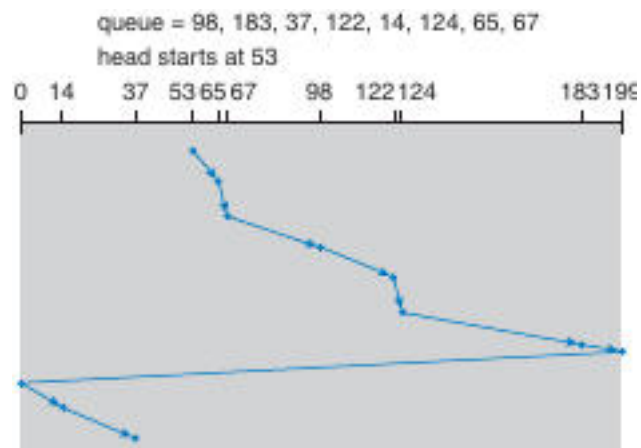
- Variants of SCAN/C-SCAN that **stop at the last request** in the current direction rather than traveling to the physical disk end.



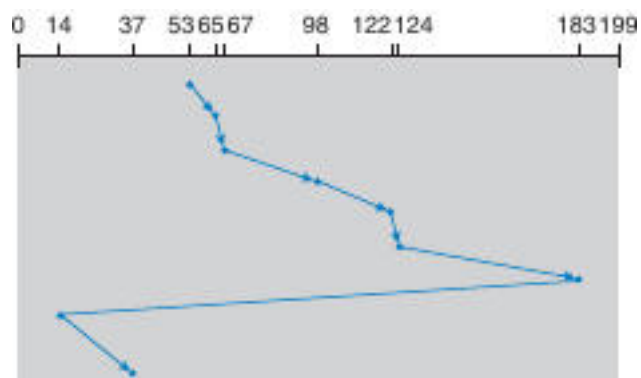
The plot shows the large back-and-forth movement typical of FCFS scheduling.



Here the head follows the nearest-request path, dramatically reducing travel distance.



The diagram visualizes the SCAN algorithm's "elevator" motion across cylinders.



The C-LOOK strategy only travels as far as the farthest request before resetting.

Selecting an Algorithm

Factor	Preferred Algorithm
--------	---------------------

Low request density (few pending I/Os)	FCFS (overhead negligible)
High locality, moderate load	SSTF (good average seek reduction)
Need for fairness, uniform wait	SCAN / LOOK
Predictable latency for real-time workloads	C-SCAN / C-LOOK
SSD-based storage	Simple FCFS or No-op (merges adjacent requests)

- The OS may also delegate scheduling to the **disk controller**, which can internally reorder requests for both seek and rotational latency optimization.
- For **demand paging**, writes are often given higher priority than reads to avoid exhausting free pages.

Disk Management: Formatting, Bad-Block Handling, ECC

- **Low-level formatting** writes a **sector header**, **data area**, and **trailer** to every sector.
 - Trailer stores **ECC (Error-Correcting Code)** and other control information.
- During normal I/O, the controller **calculates ECC** for the data written; on read, it recomputes ECC and compares it with the stored value.
 - **Soft error** → ECC can correct a few flipped bits.
 - **Hard error** → mismatch indicates a potentially **bad sector**; the sector is marked unusable and a spare sector is substituted.
- Modern drives ship with **pre-allocated spare sectors** to replace defective ones transparently.

Impact of SSDs on Disk Scheduling

- Since SSDs have **no mechanical seek**, the primary scheduling concern (minimizing head movement) disappears.
- Linux’s **Noop scheduler** treats requests as a FIFO queue, only **merging adjacent logical blocks** to reduce write amplification.
- For SSDs, **read latency is uniform**, but **write latency varies** with block erasure patterns; some SSD-specific schedulers prioritize writes accordingly.

Key Metrics Summary

Metric	Magnetic HDD	SSD
Seek time	2–10 ms (mechanical)	≈ 0 ms
Rotational latency	2–5 ms (depends on RPM)	N/A
Transfer rate	100–200 MB/s (depends on interface)	300 MB/s->5 GB/s (PCIe NVMe)
Power consumption	5–10 W (spindle)	2–5 W
Cost / GB	Low	Higher
Typical scheduling	FCFS, SSTF, SCAN, C-SCAN, LOOK	FCFS / Noop (merge only)

Disk Formatting & Partitioning

Low-level formatting – The process by which the disk controller writes a header, data area, and trailer to each sector, establishing the physical layout of the disk.

- **Sector size** can be chosen from a few standard values (e.g., 256 B, 512 B, 1 KB).
 - Larger sectors ⇒ fewer total sectors per track, but less overhead from headers/trailers, leaving more space for user data.
 - After low-level formatting the OS performs **logical formatting** (file-system creation):
 1. **Partitioning** – Divides the disk into one or more groups of cylinders; each partition can be treated as an independent logical disk.
 2. **File-system structures** – Allocates maps, free-space tables, and an initial empty directory.
 - Many file systems group blocks into **clusters** so that I/O is performed on larger, sequential chunks, improving throughput and reducing random-access penalties.
 - Some OSes expose a **raw disk** (a large sequential array of logical blocks without any file-system metadata). Raw I/O is used by database systems and other applications that need precise control over the physical location of their data.
-

Boot Block & System Startup

Boot block – A small program stored in read-only memory (ROM) that runs immediately after power-up, loads a more complete bootstrap loader from disk, and then starts the operating-system kernel.

- The ROM boot code is immutable, protecting it from viruses.
 - Because ROM cannot be altered, most systems store only a tiny **bootstrap loader** in ROM; the loader reads the real bootstrap program from the disk's **boot blocks**.
 - On disks using the **master boot record (MBR)** layout:
 - The first sector (the MBR) contains both boot code and a **partition table** that lists all partitions and identifies the **boot partition**.
 - After the MBR is executed, the boot loader reads the **boot sector** of the selected partition and continues the OS start-up (loading drivers, system services, etc.).
-

Bad Blocks & Error Handling

Bad block – A sector that cannot reliably store data due to physical damage or manufacturing defects.

- **Detection** – The controller uses ECC on each sector; a read that fails ECC correction reports the sector as bad.
 - **Sector sparing** – At factory low-level format the controller reserves spare sectors. When a bad block is discovered, the controller remaps the logical address to a spare sector.
 - **Sector slipping** – An alternative where the controller shifts all subsequent logical sectors up by one position, moving the defective sector to the end of the cylinder. This preserves the logical ordering but can invalidate OS disk-scheduling optimizations.
 - Bad-block handling may be **manual** (e.g., Linux badblocks run during formatting) or **automatic** (controller-maintained bad-block list).
 - **Soft errors** can be corrected on the fly; **hard errors** result in lost data and typically require restoration from backup.
-

Swap-Space Management

Swap space – Disk area used as an extension of main memory, holding pages that have been evicted from RAM.

Uses

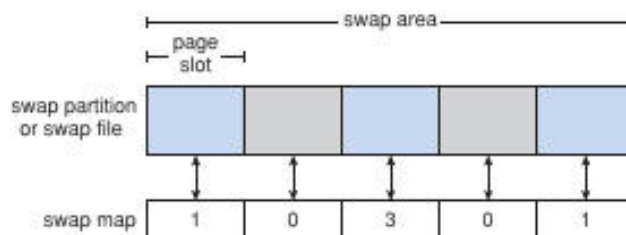
- Stores entire process images for classic swapping (rare today).
- Holds individual **anonymous pages** (stack, heap, uninitialized data) that lack a backing file.
- Provides a safety margin; insufficient swap can cause process termination or system crash.

Location Options

Location	Advantages	Disadvantages
Dedicated swap partition (raw)	Minimal file-system overhead, contiguous allocation, better performance	Requires partitioning; resizing needs repartitioning or additional partitions
Swap file inside a regular file system	Easy to create, resize, and manage with standard tools	Additional directory-lookup overhead, possible fragmentation, slightly slower I/O

Management Details (Linux example)

- Swap area consists of fixed-size **page slots** (typically 4 KB).
- A **swap map** (array of counters) tracks each slot:
 - 0 → free slot
 - >0 → occupied; the counter shows how many processes reference the swapped page (useful for shared pages).



The diagram shows a swap region divided into page slots, with a corresponding swap map indicating which slots are in use. Arrows illustrate the mapping between map entries and their page slots.

- Modern systems (Linux, Solaris) allocate swap **on demand**: a page is assigned a slot only when it actually needs to be written out, reducing unnecessary I/O.
- Swap can be spread across multiple disks to increase I/O bandwidth, similar to RAID striping for swap partitions.

RAID Structure & Reliability/Performance

RAID (Redundant Array of Independent Disks) – A set of techniques that combine multiple disks to improve reliability, performance, or both.

Reliability Through Redundancy

- **Mean Time To Failure (MTTF)** of a single disk $\approx 100\,000\text{h}$.
- In an array of N disks, the MTTF for *any* disk failing is roughly $100\,000\text{h} / N$.
- **Mirroring (RAID 1)** stores two copies of each block:
 - Failure of one disk does not cause data loss; the system continues using the surviving copy.
 - Overall system reliability $\approx \text{MTTF}^2 / (2 \times \text{MTTR})$.
 - Example: two disks each with MTTF = 100 000h and mean repair time (MTTR) = 10h yields a projected lifetime of $\sim 57\,000$ years before data loss.
- Redundancy does not protect against correlated failures (e.g., power loss, natural disasters) or simultaneous failures during repair.

Performance Via Parallelism

- **Disk mirroring** can double read throughput because either disk may satisfy a read request.
- **Striping (RAID 0)** spreads data across N disks, multiplying transfer rate by N for large sequential accesses.
- **Bit-level striping** distributes individual bits of each byte across disks; **block-level striping** (more common) distributes whole blocks.
- Combining striping with redundancy yields hybrid levels (e.g., RAID 5, RAID 6) that provide both improved throughput and fault tolerance.

RAID Levels Overview

Level	Redundancy	Striping	Parity Placement	Typical Overhead
0	None	Yes (striped)	N/A	0 % (no extra disks)
1	Full mirror	No	N/A	100 % (duplicate disks)
2	ECC (memory-style)	Yes	Dedicated ECC disks	33 % (requires ≥ 3 disks)
3	Bit-interleaved parity	Yes	Single dedicated parity disk	12.5 % (1 parity disk)
4	Block-interleaved parity	Yes	Single dedicated parity disk	12.5 %
5	Distributed block parity	Yes	Parity spread across all disks	12.5 % (effective)
6	Dual parity (P + Q)	Yes	Two distributed parity blocks	25 %

- **RAID 5** offers a good balance: one parity block per stripe, allowing any single-disk failure to be rebuilt without loss of data.
- **RAID 6** adds a second parity block, tolerating two simultaneous disk failures at the cost of additional overhead.

Write Performance Considerations

- Parity-based levels (3, 4, 5, 6) incur extra write latency because the parity must be read, updated, and written for every data write.
 - Hardware RAID controllers often include a dedicated cache (sometimes NVRAM) to absorb write bursts and compute parity off-CPU, mitigating the penalty.
-

Integration with Earlier Concepts

- The **swap-space** mechanisms described here rely on the same **page-fault handling** discussed in the TLB and page-replacement sections; a page fault may cause a page to be written to swap if no free frame exists.
- **Bad-block remapping** can affect the effectiveness of disk-scheduling algorithms (e.g., SSTF, SCAN) introduced earlier, because logical-to-physical address translations may change dynamically.
- **RAID** improves the I/O characteristics that influence **working-set** and **TLB reach** calculations: higher throughput and lower latency can reduce the frequency of page faults for I/O-bound workloads.

RAID Levels Overview

RAID 4 – Block-interleaved parity (dedicated parity disk)

RAID 4 uses **block-level striping** like RAID 0, but stores a **single parity block** on a separate disk for each stripe.

- **Read access** – A block read touches only the data disk that holds the requested block, allowing multiple reads to proceed in parallel on the other disks.
- **Write access** –
 - Large sequential writes can be issued to data disks **and** the parity disk simultaneously, giving high throughput.
 - Small, independent writes must perform a **read-modify-write (RMW) cycle**:
 1. Read the old data block and the old parity block.
 2. Compute the new parity.
 3. Write the new data block and the new parity block.→ Four disk accesses for a single small write.
- **WAFL** (the NetApp file system) adopts RAID 4 because a newly added disk can be initialized with all-zero blocks; the parity does not change, so the RAID set remains correct without re-striping.

RAID 5 – Distributed parity

RAID 5 spreads both **data** and **parity** blocks across all disks in the array, rotating the parity position for each stripe.

- Example with five disks: the parity for stripe n resides on disk $(n \bmod 5)$, while the remaining four disks hold the data blocks.
- Advantages over RAID 4:
 - No single “parity disk” becomes a bottleneck.
 - Load is balanced across all disks, improving write performance for small writes.
- RAID 5 is the **most common** configuration for general-purpose servers because it offers a good compromise between capacity, reliability, and throughput.

RAID 6 – Double parity (P+Q)

RAID 6 adds a second independent parity block (often called **Q**) using Reed–Solomon codes, allowing the array to survive **two simultaneous disk failures**.

- Each stripe contains **data**, **P parity**, and **Q parity** distributed over all disks.
- Write penalty is higher than RAID 5 because two parity calculations and writes are required, but the extra redundancy greatly reduces the risk of data loss during rebuilds.

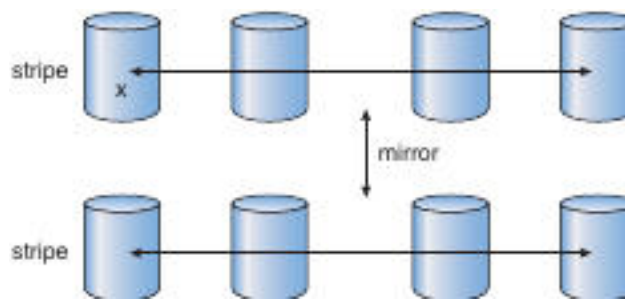
- Suitable for large arrays where the probability of a second failure during rebuild is non-negligible.

RAID 0+1 and RAID 1+0 (RAID 10) – Mirroring + Striping

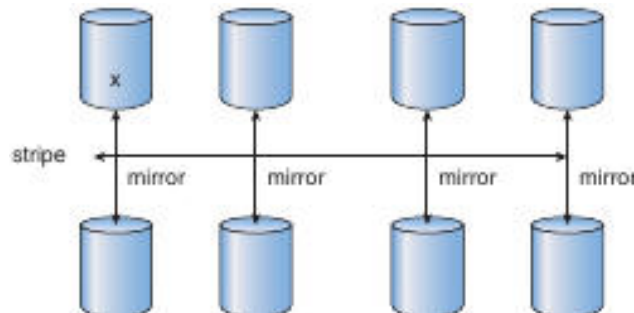
RAID 0+1 first stripes data across a set of disks, then mirrors the entire stripe set.

RAID 1+0 mirrors disks in pairs first, then stripes across the mirrored pairs.

- Both schemes provide **high performance** (from striping) and **high reliability** (from mirroring).
- Failure characteristics differ:
 - In RAID 0+1, a single disk failure renders the entire striped set unavailable; the system falls back to the mirrored copy.
 - In RAID 1+0, each mirrored pair can lose **one** disk without affecting the stripe, so up to $N/2$ disks may fail (as long as they are in different pairs).



The diagram shows a striped set of cylinders (top row) mirrored by a second identical row (bottom). The “x” marks the start of the stripe; the vertical arrow labeled “mirror” indicates the mirroring relationship.



Four striped cylinders (top) are each mirrored by a disk below, illustrating the RAID 10 layout that combines striping for performance with mirroring for redundancy.

RAID Implementation Options

Implementation Layer	Typical Characteristics
Software RAID (volume-manager)	Implemented in OS; flexible but slower because all parity calculations occur on the host CPU.

Host Bus Adapter (HBA) RAID	Hardware on the host controller; offloads parity work from the CPU, moderate cost, limited to disks attached to that HBA.
Dedicated storage-array hardware RAID	Full RAID engine inside the array; offers the best performance and many advanced features (snapshots, replication).
SAN-level virtualization	RAID performed by a disk-virtualization device between hosts and storage; enables shared RAID sets across multiple servers.

- **Snapshots** – point-in-time, read-only views of a volume (e.g., WAFL or ZFS snapshots).
- **Replication** – automatic duplication of writes to a remote site:
 - *Synchronous* – write must be committed locally **and** remotely before completing.
 - *Asynchronous* – writes are buffered locally and sent in batches; faster but can lose recent data if the remote site fails.
- **Hot spare** – a standby disk that is automatically incorporated into the array when a member fails, allowing immediate rebuild without administrator intervention. Multiple hot spares can protect against more than one simultaneous failure.

Selecting a RAID Level

Choosing a RAID level involves balancing **performance**, **reliability**, **capacity overhead**, and **rebuild time**.

RAID Level	Typical Use	Pros	Cons
0 (striping)	High-throughput, non-critical workloads	Max bandwidth, no overhead	No redundancy
1 (mirroring)	Mission-critical data requiring fast recovery	Immediate read/write redundancy, simple rebuild	2× storage cost
4	Environments needing easy disk addition (e.g., WAFL)	Simple parity, constant-time rebuild for added disks	Write-modify-write penalty for small writes
5	General-purpose servers, large data stores	Good space efficiency ($\approx 1 / (N+1)$ overhead), balanced load	Slower small-write performance, long rebuild times for large arrays
6	Very large arrays where two-disk failure tolerance is required	Tolerates two simultaneous failures	Higher write overhead, more CPU for Reed-Solomon calculations
0+1 / 1+0 (RAID 10)	Databases, virtualization hosts needing both speed and fault tolerance	Excellent read/write performance, can survive multiple failures (depending on layout)	2× storage cost, complex failure semantics

- **Rebuild time** matters for high-availability systems: RAID 1 rebuilds are fast (simple copy), while RAID 5/6 may take hours on large disks because all surviving disks must be read to regenerate parity.
- **Space overhead**: larger parity groups (more data per parity block) reduce overhead but increase the probability that a second disk fails before the first is rebuilt.

Extensions of RAID Concepts

- **RAID for tape arrays** – Robots with multiple tapes can be treated as a RAID set; parity allows reconstruction if a single tape fails, improving backup reliability.
- **RAID for broadcast** – Data streams are split into small units; parity units are transmitted alongside. If any unit is lost, the original data can be recovered using the parity information.
- **RAID-like striping across SSDs** – Although SSDs have negligible seek time, spreading data across several devices can still increase aggregate bandwidth.

RAID Limitations & Data Integrity

RAID protects **physical media failures** but does **not** guard against logical corruption, software bugs, or mis-directed pointers.

- File-system metadata can become inconsistent even on a healthy RAID set.
- **ZFS** mitigates this by storing **checksums** for every data and metadata block; a bad block is detected by a mismatched checksum and automatically repaired from a good mirror.
- **InServ** (3Par) uses a *chunklet* model: disks are divided into 256-MB pieces that can belong to multiple RAID levels simultaneously, enabling flexible volume resizing and on-the-fly RAID-type changes.

Stable-Storage Implementation

To guarantee that a write either **completes fully** or **has no effect**, the system can use a two-copy algorithm:

1. Write the new data to a **first physical block**.
2. After the first write succeeds, write the same data to a **second physical block**.
3. Declare the operation successful only after the second write finishes.

During recovery:

- If both copies match and are error-free → no action needed.
- If one copy shows a detectable error → replace it with the good copy.
- If both copies are error-free but differ → choose one (e.g., the second) as the correct value.
- Extending to **N copies** further reduces the chance of total loss, though two copies are often sufficient for practical “stable storage”.
- Modern arrays accelerate this process by using **NVRAM caches** (battery-backed) as a fast, non-volatile staging area; writes to NVRAM complete quickly, and the cache later flushes data to disk, providing both speed and durability.

These notes build on earlier concepts such as working-set size, I/O bandwidth, and fault handling, illustrating how RAID choices directly affect overall system performance and reliability.

Disk-Scheduling Algorithms

Performance factors

- **Seek time** – time for the actuator to move the read/write head to the target cylinder.

- **Rotational latency** – waiting for the desired sector to rotate under the head.
- **Bandwidth** – amount of data transferred once the head is positioned.

The total response time for a request $\approx$ seek time + rotational latency + transfer time.

Non-uniform request distribution

Real workloads often exhibit “hot spots” (e.g., cylinder storing file-system metadata) that receive a large fraction of I/O requests.

a. Which classic algorithm suits a hot-spot workload?

- **SCAN / C-SCAN** tend to be advantageous because the head repeatedly sweeps across the cylinder range, giving the hot spot many chances to be serviced as the sweep passes over it.
- **SSTF** can also perform well if the hot spot lies near the current head position, but it may starve requests on the opposite end of the disk.

b. Proposed hot-spot-aware scheduler

1. **Maintain two queues** – one for the hot-spot cylinder(s) and one for all other requests.
2. **Prioritize the hot-spot queue** whenever the head is within a configurable distance d of the hot cylinder; otherwise run a standard SCAN pass on the normal queue.
3. **Dynamic adaptation** – periodically update the hot-spot set based on recent request frequencies.

This hybrid approach keeps the **even-performance** of SCAN while guaranteeing low latency for the frequently accessed cylinders.

RAID-5 Write Characteristics

a. Single-block write

1. **Read-modify-write (RMW)**: read the old data block and the old parity block, compute new parity, write both the new data block and the new parity block.
2. **I/O cost**: 4 disk operations (read data + read parity + write data + write parity).

b. Seven-block contiguous write

- The stripe can be written in one **full-stripe write**:
 - Write the 7 data blocks, compute parity on-the-fly, and write the parity block once.
 - **I/O cost**: 8 operations (7 data writes + 1 parity write).
- This reduces per-block overhead compared with seven separate single-block writes.

RAID-1 vs. RAID-5 read throughput

Access pattern	RAID 1 (mirrored)	RAID 5 (striped + parity)
Single block	Can read from either mirror $\rightarrow \approx 2\times$ read bandwidth of a single disk.	Reads come from a single data disk (parity not needed) $\rightarrow \approx 1\times$ disk bandwidth.
Contiguous blocks	Both mirrors can be read in parallel, but each block still resides on one disk $\rightarrow \approx 2\times$ bandwidth.	Data is striped across $N-1$ disks $\rightarrow \approx N-1\times$ bandwidth (significant gain for large N).

Write performance comparison

- **RAID 1**: each write updates two disks (both mirrors) → 2 writes per operation.
- **RAID 5**: single-block write incurs 4 operations (RMW), but full-stripe writes only need N writes.
- **Conclusion**: RAID 1 excels for small random writes; RAID 5 shines when writes are large and sequential.

Mixed RAID configuration (RAID 1+RAID 5)

- **Store read-intensive, small-write files** (e.g., configuration files, logs) on the **RAID 1** portion to benefit from fast, reliable reads and low-latency writes.
- **Store large, sequential files** (e.g., media, database tables) on **RAID 5** to exploit striping bandwidth and efficient large-write handling.

Reliability Metrics & MTBF

Mean Time Between Failures (MTBF) – the expected elapsed time between two consecutive failures of a component. It is expressed in **hours**.

a. Failure frequency for 1000 drives

- Each drive: MTBF = 750 000 h (≈ 85.6 years).
- System MTBF for k independent drives $\approx \text{MTBF} / k$.
- For 1000 drives:

$$\$ \text{System MTBF} = \frac{750\{, \}000 \text{ h}}{1\{, \}000} = 750 \text{ h} \approx 31 \text{ days} \$$$

Thus, on average **one drive fails roughly every 31 days**.

b. Expected lifetime of a 20-year-old drive

- If the manufacturer quotes 750 000 h MTBF, a 20-year-old drive has already operated

$$\$ 20 \text{ years} \times 365 \text{ days/year} \times 24 \text{ h/day} = 175\{, \}200 \text{ h} \$$$

- Remaining expected life $\approx 750\,000 \text{ h} - 175\,200 \text{ h} \approx 574\,800 \text{ h}$ (≈ 65.6 years).
- **Interpretation**: the MTBF figure is *statistical*; individual drives can survive much longer than the nominal MTBF.

c. 1-million-hour MTBF guarantee

- A 1 000 000 h MTBF translates to ≈ 114 years.
- For a system with many disks, the *aggregate* failure rate still scales with the number of drives (see part a).
- The guarantee mainly **communicates reliability** to the purchaser; it does not mean a single drive will actually run a century without failure.

Sector Slipping vs. Sector Remapping

Technique	How it works	Advantages	Disadvantages
Sector slipping	When a bad sector is discovered, the controller shifts all subsequent logical sectors forward by	Preserves sequential ordering of logical sectors, aiding OS disk-scheduling heuristics.	Requires moving many sectors; costly on large disks; limited number of slips per track.

	one physical sector, leaving the defective sector at the end of the track.		
Sector remapping (spare-sector replacement)	The defective sector is mapped to a pre-allocated spare sector; logical-to-physical mapping table is updated.	Fast; only the bad sector is replaced; unlimited number of replacements (up to spare pool size).	Breaks strict sequential ordering; may affect algorithms that assume contiguity.

OS Knowledge of Disk Geometry

- **Why the OS needs layout information**
 - To place frequently accessed metadata (e.g., inode tables) on *hot* cylinders, reducing average seek time.
 - To schedule I/O efficiently (e.g., grouping requests that fall on the same track or cylinder).
- **Performance gains**
 - By locating related blocks consecutively, the OS can serve multiple requests with a single seek and reduced rotational latency.
 - File-system allocators can implement *clustered allocation* (as in Windows' clustering) to exploit locality.

Disk-Scheduling Programming Assignment

Implement the following algorithms for a disk with cylinders **0–5000**:

Algorithm	Description
FCFS	Service requests in arrival order.
SSTF	Service the request nearest to the current head position.
SCAN	Move the head in one direction, servicing requests until the end, then reverse.
C-SCAN	Same as SCAN, but after reaching the end, jump back to the start without servicing on the return.
LOOK	Like SCAN but reverse direction at the last pending request instead of the physical end.
C-LOOK	Like C-SCAN but jump back to the first pending request.

Program requirements

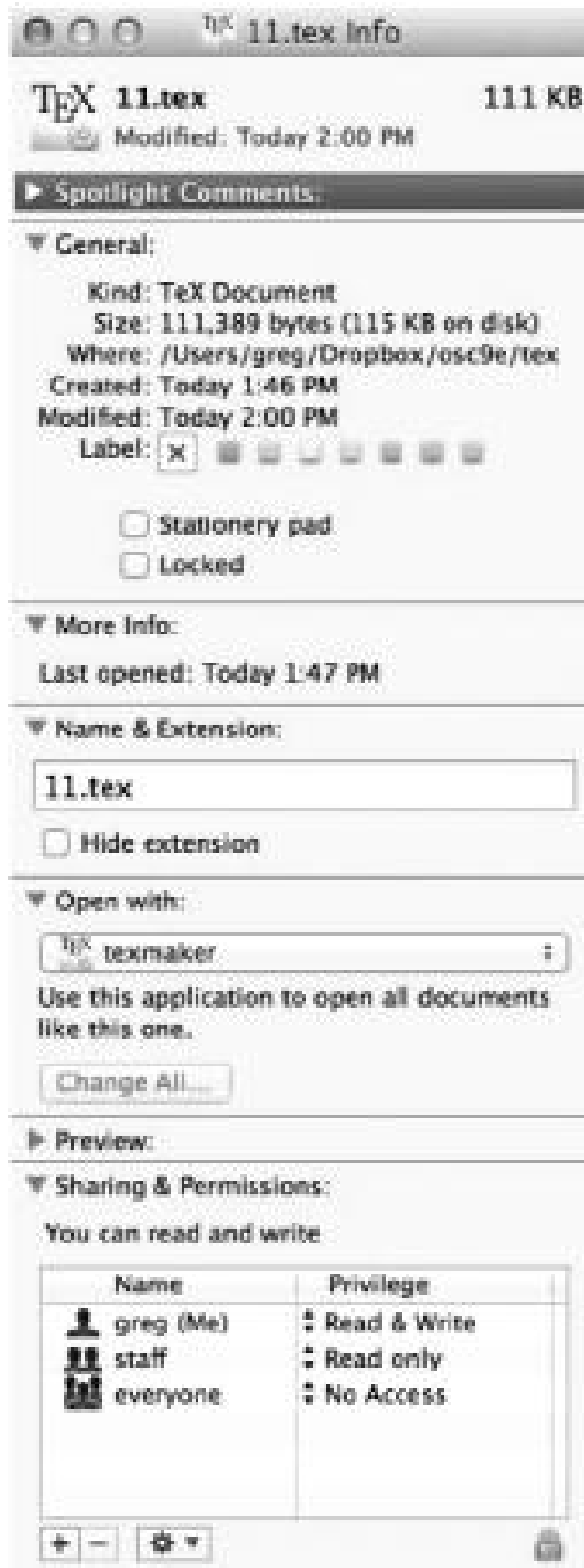
- Accept the initial head position as a command-line argument.
- Generate **1000** random cylinder requests.
- Report the **total head movement** (in cylinders) for each algorithm.

File System Fundamentals

A *file* – a named collection of related information stored on secondary storage; its logical view is presented uniformly to users and applications.

File Attributes

Attribute	Description
Name	Human-readable identifier (e.g., example.c).
Identifier (inode / UID)	System-generated unique number used internally.
Type	Indicates how the OS should treat the file (regular, directory, device, etc.).
Location	Pointer to the device and block(s) where the file's data resides.
Size	Current length in bytes (or blocks).
Protection	Access-control bits (read, write, execute) and owner/group info.
Timestamps	Creation, last modification, last access times.
Extended attributes (optional)	Checksums, character encoding, creator application, etc.



The screenshot shows a macOS file-info dialog for “11.tex”, displaying type, size, creation/modification dates, and a detailed permissions table. This visual illustrates the concrete attributes listed above.

Directory Structure

- Each directory entry stores the **file name** and its **unique identifier** (inode).
- The identifier links to the file’s metadata (size, timestamps, block locations) stored elsewhere on disk.

Basic File Operations

Operation	Key OS actions
Create	Allocate a directory entry, reserve initial blocks (often none), update the directory.
Write	Locate file via name → directory → inode; use the write pointer (per-process file offset) to place data; update pointer and possibly allocate new blocks.
Read	Similar lookup; use the read pointer to fetch data from the current offset; update pointer after the read.
Reposition (seek)	Adjust the per-process file-position pointer without I/O; can move forward or backward.
Delete	Remove the directory entry, release all allocated blocks back to the free-space pool, decrement link count if hard links exist.
Truncate	Set file length to zero (or a new length); release any blocks beyond the new size while preserving metadata.

- **Per-process file-position pointer** is stored in the **process-local open-file table**; it eliminates the need to pass an explicit offset on each I/O call.
- The OS keeps an **open-file table** (system-wide) that holds immutable file metadata (disk location, size, timestamps) and a **reference count** indicating how many processes have the file open.

File Locking

- **Lock types**
 - *Shared lock* (reader) – multiple processes may hold it simultaneously.
 - *Exclusive lock* (writer) – only one process may hold it; blocks all others.
- **Lock enforcement**
 - **Mandatory locking** – the OS kernel blocks any access that conflicts with an existing lock.
 - **Advisory locking** – the OS does not enforce; cooperating processes must check for locks themselves.
 - Windows typically uses **mandatory** locks; UNIX systems favor **advisory** locks.
- **Java example (excerpt)**

```
RandomAccessFile raf = new RandomAccessFile("file.txt", "rw");
FileChannel ch = raf.getChannel();
FileLock exclusiveLock = ch.lock(0, raf.length()/2, false); // exclusive on first half
```

```
// ... modify data ...
exclusiveLock.release();

FileLock sharedLock = ch.lock(raf.length()/2+1, raf.length(), true); // shared on second half
// ... read data ...
sharedLock.release();
```

- **Best practices**
 - Acquire the lock **before** performing I/O on the protected region.
 - Release the lock promptly to avoid deadlock.
 - For mandatory locking, ensure exclusive locks are held only for the minimal critical section.

File Types & Extensions

- **Extension-based typing** – the part of a filename after the final period (e.g., .docx, .exe).
- **Magic numbers** – OS-level signatures stored at the start of a file (used by UNIX to identify binaries, PDFs, etc.).
- **Creator attribute** – records the application that created the file; used by GUIs to launch the appropriate program on double-click.

Extension	Typical Type	Example Use
.c, .cpp, .java	Source code	Compiled by a language-specific compiler
.exe, .com	Binary executable	Directly runnable by the OS
.sh	Shell script	Executed by a command interpreter
.txt, .md	Plain text	Viewed/edited with any editor
.pdf, .docx	Document	Opened by PDF viewer or word processor
.jpg, .png	Image	Rendered by image viewers
.zip, .tar, .rar	Archive	Extracted by compression utilities
.mp3, .mp4	Multimedia	Played by media players
.so, .dll	Library	Dynamically linked at runtime

- Extensions are **hints**; the OS may ignore them, but applications often rely on them to select appropriate handlers.

File Structure

- Certain file types (e.g., executables, object files) have **mandatory internal layouts** that the loader must understand (header, segment tables, entry point).
- The OS may provide **system-supported structures** (e.g., ELF for Unix, PE for Windows). Supporting multiple structures increases kernel size and complexity.
- When the OS knows a file's internal format, it can:
 - **Validate** the file before execution.
 - **Map** only the necessary segments into memory (demand paging).
 - **Apply** special caching or prefetching strategies (e.g., keep the code segment hot).

These notes expand on earlier concepts such as [TLB reach](#), [working-set models](#), and [RAID reliability](#), tying them to concrete disk-scheduling, RAID performance, and file-system mechanisms.

File Types & OS Support

Definition: *File type* – the classification the operating system uses to interpret a file’s contents (e.g., ASCII text, executable binary).

- Some OSes restrict files to a few built-in types.
- **Problem:** Encrypted data appears as random bits, matching neither plain-text nor executable formats.
- To store encrypted files, applications may need to **bypass** the OS file-type checks or rely on a more generic “byte-stream” type.

Minimal File Structure (UNIX-style)

Definition: *Byte-stream file* – a file treated as an uninterrupted sequence of 8-bit bytes, with no imposed structure.

- The OS does **not** interpret the bytes; each application parses the data it reads.
- Guarantees at least one universal structure so that programs (including loaders) can be executed.
- Provides **maximum flexibility** but places the burden of format handling on user programs.

Internal File Structure & Block Management

- Disks perform I/O in fixed-size **blocks** (physical sectors). Typical block size: **512 B**.
- **Logical records** (e.g., a line of text, a database row) often have variable length and rarely line up exactly with block boundaries.

Packing Strategies

Strategy	How it works	Pros	Cons
Application-level packing	Program groups several logical records into one block before issuing a write.	Full control over layout; can tailor to record size.	Requires extra code in every app.
OS-level packing	File system automatically assembles bytes into blocks.	Transparent to applications.	May waste space if records don’t fit neatly.

- **Internal fragmentation** occurs when the last allocated block is only partially used.
 - Example: 1949-byte file → 4 × 512 B blocks = 2048 B allocated → **99 B** wasted.
 - Larger block sizes increase potential waste.

File Access Methods

10.2.1 Sequential Access

Definition: *Sequential access* – reading or writing records in the order they appear in the file, moving a file pointer forward after each operation.

- Operations: `read_next()` (read current record, advance pointer), `write_next()` (append, advance).
- Supports simple rewind to the beginning; some systems allow moving backward by a single record.

10.2.2 Direct (Random) Access

Definition: *Direct access* – reading or writing a specific logical record or block without regard to its position relative to previous operations.

- Calls include `read(n)` and `write(n)`, where *n* is a **relative block number** (0-based from the file's start).
- Enables fast lookup for large data sets (e.g., airline reservation systems: block 713 holds seats for flight 713).

Simulating Direct Access on a Sequential File

```
int cp = 0;           // current position
read(cp);             // read current block
cp++;                 // advance manually
```

- Inefficient: requires scanning through intermediate blocks to reach the desired position.

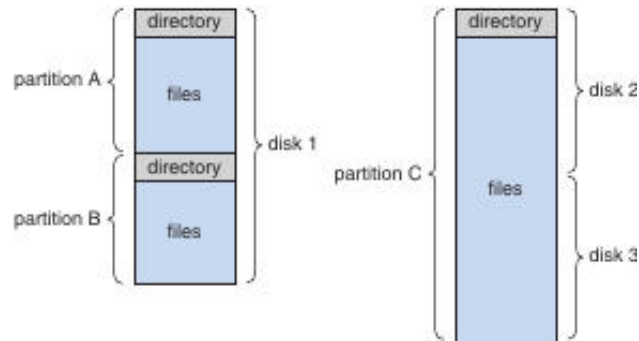
10.2.3 Indexed / Hierarchical Access

- An **index file** stores pointers (disk block numbers) to data blocks.
- Multi-level indexing (primary → secondary → tertiary) reduces the number of I/O operations needed to locate a record.
- Example: IBM's **ISAM** (Indexed Sequential Access Method) uses a master index → secondary index → data blocks, allowing a record to be found with at most two direct reads.

Directory & Disk Structure

10.3.1 Volumes, Partitions, & RAID

- A storage device can be **partitioned** into multiple logical sections, each hosting an independent file system (e.g., partitions A, B, C on Disk 1).
- Multiple devices can be combined into a **RAID set** for redundancy or performance.
- The logical collection of a file system on a device (or RAID set) is called a **volume**.



The diagram shows Disk 1 split into three partitions (A, B, C) each with its own directory and files, while Disk 2 & 3 are combined as a single entity with a shared directory. This visualizes how storage can be sliced into volumes and how each volume maintains its own file hierarchy.

- **Backup strategy:** periodically copy the entire volume (or selected files) to magnetic tape to protect against failure.

10.3.2 Directory Overview

Definition: *Directory* – a table that maps file names to metadata (size, location, type, etc.).

Common directory operations:

- **Search** – locate an entry by name or pattern.
- **Create** – add a new entry.
- **Delete** – remove an entry and free its blocks.
- **List** – enumerate all entries in a directory.
- **Rename** – change the name (and optionally the path).
- **Traverse** – walk the entire directory tree (useful for backups).



Directory Structures

10.3.3 Single-Level Directory

- All files reside in one flat table.
- **Limitation:** file names must be globally unique; name collisions become inevitable as the number of users grows.

10.3.4 Two-Level Directory

- **Master File Directory (MFD):** maps *user IDs* to **User File Directories (UFDs)**.
- Each UFD contains the files belonging to a single user.
- Allows duplicate file names across different users (e.g., prog2.c can exist in both User A and User B).

Path Naming Example

- Absolute path: /userb/test.txt → root → userb UFD → test.txt.
- Relative path (from User B's current directory): test.txt.

10.3.5 Tree-Structured Directories

- Generalization of the two-level model to an arbitrary-depth hierarchy.

- **Root directory** → subdirectories → ... → leaf files.
- Each entry includes a flag indicating *file* (0) or *subdirectory* (1).

Path Types

Type	Syntax	Example
Absolute	Begins at root (/).	/spell/mail/prt/first
Relative	Starts from current directory.	prt/first (when cwd = /spell/mail)

- **Current directory** (cwd) is maintained per process; system call `chdir()` changes it.

Deletion Policies

- **Strict**: refuse to delete a non-empty directory; user must remove contents first (recursively).
- **Recursive (e.g., UNIX `rm -r`)**: delete a directory and all of its descendants in one operation – convenient but risky.

Example File-System Types (Solaris)

FS Type	Characteristics
tmpfs	In-memory, volatile; cleared on reboot.
objfs	Exposes kernel objects (e.g., contracts) as files.
ctfs	Virtual FS for process contracts and system state.
procfs	Provides a view of running processes.
zfs / ufs	General-purpose persistent file systems.
lofs	Loop-back FS; maps one directory tree onto another.

- Different FS types coexist on the same system, each mounted on its own **mount point** (e.g., /proc, /dev).

Index & Relative Files Example

- **Index file** stores the first key of each data block (e.g., first UPC in a block).
- **Relative file** records are addressed by a *relative block number* (0-based) rather than an absolute disk address.

Data file	Block size	Records per block
Retail-price list	1024 B	64 (16-byte records)
Total records	120 000	→ 2 000 blocks (≈ 2 MB)
Index size	20 000 B	→ fits easily in memory

- A **binary search** on the index yields the correct block; a second binary search on the secondary index (if present) narrows it further, finally a sequential scan within the block locates the exact record.

Practical Considerations

- **Block size selection** balances I/O efficiency (larger blocks → fewer seeks) against internal fragmentation (larger blocks → more wasted bytes).

- **Choosing an access method** depends on workload:
 - Sequential for log files, compilers, editors.
 - Direct for databases, reservation systems, random-access queries.
 - Indexed for large sorted files where fast lookups are required.
- **Directory design** impacts name management, security, and scalability:
 - Single-level → simple but impractical for multi-user environments.
 - Two-level → isolates users, reduces name collisions.
 - Tree-structured → flexible organization, supports deep hierarchies and fine-grained permissions.

These concepts tie back to earlier discussions on **working sets**, **TLB reach**, and **paging**: the way files are organized and accessed directly influences I/O patterns, which in turn affect page-fault rates and overall system performance.

Shared Files & Directories

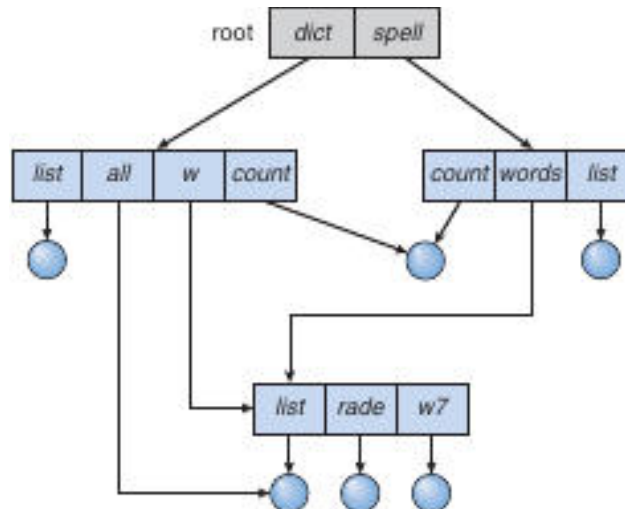
A shared file or directory is a single actual entity that appears in multiple places of the directory hierarchy. Changes made through any reference are instantly visible through all other references.

- **Copy vs. Share**
 - *Copy*: Two independent files; modifications to one do **not** affect the other.
 - *Shared (link)*: Only one physical file; every programmer sees the same data.
 - Particularly useful for **subdirectories**: a new file created in one shared subdirectory automatically appears in every other location where that subdirectory is linked.
-

Directory Structures: Tree vs. Acyclic Graph

A tree-structured directory is a directed graph with no cycles; each node (directory) has exactly one parent.

- **Limitations of a pure tree**
 - Cannot represent a file or subdirectory that belongs to more than one parent.
- **Acyclic-graph directory** (graph with no cycles but allowing multiple parents)
 - Enables sharing of files and directories without duplication.
 - Still avoids the complications of true cycles.



The diagram shows a hierarchy that can be traversed without forming a cycle, demonstrating how a directory-graph can share sub-structures.

Links (Hard and Symbolic)

A link is a directory entry that contains a pointer (path name) to another directory entry rather than the data itself.

- **Implementation styles**
 1. **Absolute link** – stores the full path to the target (e.g., /home/alice/project/file.c).
 2. **Relative link** – stores a path relative to the link's location (e.g., ../shared/lib.h).
- **Resolution process**
 1. The OS reads the link entry.
 2. It follows the stored path to locate the real file.
 3. The link is treated as an *indirect pointer*; the underlying file is not duplicated.
- **Identification** – Links are marked by a special type flag or a recognizable format in the directory entry.
- **Effect on traversal** – When the system walks the directory tree, it **ignores** link entries to preserve the acyclic property of the logical structure.

Duplicate Directory Entries vs. Links

- **Duplicate entries**: Two identical directory records point to the same data block; they are indistinguishable from the original.
- **Links**: The link record is *distinct* from the original entry, making it easy to recognize and manage.
- **Consistency challenge**: With duplicate entries, updating one copy does not automatically update the other, leading to stale data; links avoid this problem because there is only one underlying file.

Reference Counting & Deletion

Reference counting tracks how many directory entries (hard links) refer to a given file.

- Each file (inode) stores a **reference count**.
- **Creation of a hard link** increments the count; deletion decrements it.
- When the count reaches **zero**, the OS knows the file's storage blocks can be reclaimed.
- **Aliasing problem**: A single file can have multiple absolute path names; programs must be aware that different names may refer to the same data.

Deletion semantics

Situation	Effect
Delete a link only	Underlying file remains; count is decreased.
Delete the last link (count → 0)	File's disk space is deallocated.
Delete a symbolic link	Only the link entry disappears; the target file is untouched.

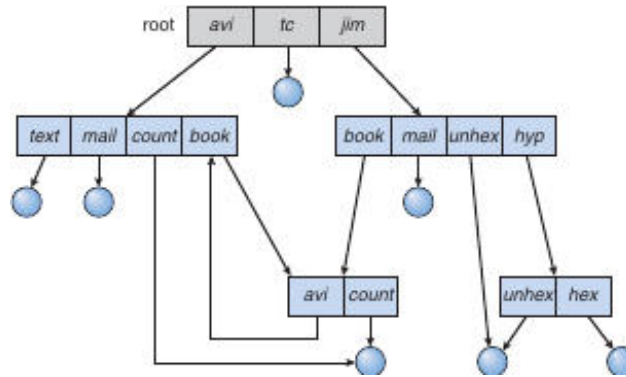
Garbage Collection for Cyclic Structures

- Allowing **cycles** in the directory graph creates situations where reference counts never drop to zero even though the files are unreachable.
- **Mark-and-sweep garbage collection**:
 1. **Mark** all reachable files by traversing from root directories.
 2. **Sweep** everything unmarked and reclaim its space.
- This process is **expensive** (especially on disk-resident graphs) and therefore most systems prefer to *prevent cycles* rather than rely on garbage collection.

Mounting File Systems

Mounting attaches an external file-system to a point in the existing directory hierarchy, making its contents accessible.

- **Mount point**: An existing directory where the new file-system will appear (e.g., /home).
- **Mount operation steps**
 1. Specify **device name** (e.g., /dev/sda1) and **mount point**.
 2. Optionally provide the **file-system type**; some OSes infer it automatically.
 3. The kernel reads the device's superblock to verify a valid file-system.
 4. The mount table is updated, linking the device's root to the chosen mount point.
- **Over-mounting**: If a directory already contains files, mounting a new file-system on it hides the original contents until the file-system is unmounted.
- **Multiple mount points**: A file-system may be mounted at several locations simultaneously (or restricted to a single mount, depending on OS policy).



The picture visualizes a typical tree-structured file system that can serve as a mount point for additional volumes.

File Sharing Overview

- **Why share?** Collaboration and avoidance of duplicated copies.
- **Access control attributes** (owner, group, others) determine who may read, write, or execute a shared object.
- In multi-user environments the OS must mediate **permission checks** for every operation, using the file's stored owner/group IDs and the requesting user's credentials.

Remote File Systems & the Client-Server Model

A remote file system lets a client machine mount and use files that physically reside on a server.

- **Client:** Issues file-system calls (open, read, write) on behalf of the user.
- **Server:** Exposes a directory tree and enforces access control.
- **Authentication challenges**
 - Simple schemes (e.g., NFS UID matching) can be spoofed.
 - More secure methods use encrypted keys or Kerberos-style tickets.
- **NFS example**
 - Stateless protocol: the server does not keep per-client session state; each request carries the user's UID/GID.
 - A file is opened, operated on, and closed in a single request; the server trusts the UID supplied.
 - Because the protocol is stateless, it is resilient to crashes but vulnerable to forged requests.
- **Mounting remote FS:** The client performs a mount operation similar to local mounting, after which remote paths become part of the local namespace (e.g., /mnt/nfs).

Consistency Semantics

Consistency semantics define when a write performed by one user becomes visible to other users of the same file.

- **UNIX semantics** (local files)
 - Writes to an open file are **immediately visible** to any other process that has the same file open.

- **Stateless NFS semantics**
 - Because the server does not retain client state, a write is considered complete once the server acknowledges it.
 - Subsequent reads from any client will see the new data, but the lack of locking can lead to race conditions.
- **Andrew File System (AFS)** provides stronger guarantees (e.g., callbacks that invalidate client caches when a file changes).
- Consistency models are crucial for **performance vs. correctness trade-offs** in distributed environments where network latency and disk I/O are significant.

Session Semantics (Andrew File System)

Session semantics describe how changes made by one user become visible to others when a file is opened, closed, or both.

- **Write visibility:**
 - Writes to an open file are **not** immediately visible to other users who already have the same file open.
 - Changes become visible **only after the file is closed** by the writer; subsequently opened instances will see the updated image.
- **Result:** A single file can temporarily have **multiple images** (different versions) co-existing, allowing concurrent reads and writes without immediate synchronization delays.

This contrasts with the **UNIX semantics** discussed earlier, where an open file is treated as a single exclusive resource and any write is instantly reflected for all processes sharing the file descriptor.

Immutable-Shared-File Semantics

An *immutable shared file* is a file that **cannot be renamed** and **cannot be altered** after its creation.

- **Key properties:**
 1. **Name stability:** The filename permanently identifies the content.
 2. **Read-only sharing:** All users may read the file, but no one can modify it.
- **Advantages in distributed systems:**
 - Simplifies consistency because the file never changes after publication.
 - Reduces the need for complex synchronization mechanisms; the sharing discipline is strictly *read-only*.

Protection Overview

Protection ensures that stored information is safe from **hardware failures** (reliability) and **unauthorized access** (security).

- **Reliability** is achieved by duplicate copies (e.g., regular backups to tape).

- **Security** is enforced through controlled access mechanisms that decide **who** may perform **what** operation on a file or directory.

Both aspects are essential for multi-user systems; a single-user laptop may rely on physical security (e.g., locking the device), whereas a multi-user server requires systematic protection policies.

Types of File Access

Access Type	Description
Read	Retrieve data from the file.
Write	Overwrite existing data.
Execute	Load the file into memory and run it.
Append	Add new data at the end of the file.
Delete	Remove the file and free its storage blocks.
List	Show the file's name and attributes (directory listing).

Higher-level operations such as **rename**, **copy**, or **move** are implemented by a sequence of these primitive accesses (e.g., a copy is a series of reads followed by writes).

Access Control Lists (ACLs)

An *access-control list* (ACL) attaches to each file or directory a list of **(user, allowed-operations)** pairs.

- **Evaluation process:** When a process requests an operation, the OS checks the file's ACL.
 - If the user appears with the required permission, the request is granted.
 - Otherwise a **protection violation** occurs and the operation is denied.

ACL Structure

Category	Meaning
Owner	User who created the file; typically has full rights.
Group	A set of users sharing similar access needs (e.g., a project team).
Universe	All other users on the system.

Most modern UNIX-like systems (e.g., Solaris) combine the classic **owner-group-world** model with optional ACL entries for fine-grained control.

Pros:

- Enables complex, per-user permissions without enumerating every possible user in the basic three-bit scheme.

Cons:

- Building and maintaining long ACLs can be tedious.
 - Fixed-size directory entries must expand to accommodate variable-length ACLs, complicating space management.
-

UNIX Permission Model (owner/group/universe)

UNIX permissions use **nine bits** (rwx for each of owner, group, and others) to grant or deny the three basic operations *read*, *write*, and *execute*.

Example listing (excerpt):

```
-rw-rw-r-- 1 pbg staff 31200 Sep  3 08:30 intro.ps
drwx----- 5 pbg staff   512 Jul  8 private/
drwxrwxr-x 2 pbg staff   512 Aug  3 14:13 project/
```

- The first character indicates file type (- regular file, d directory).
- The next three characters are the **owner** permissions, followed by **group**, then **universe**.

Permission bits definition

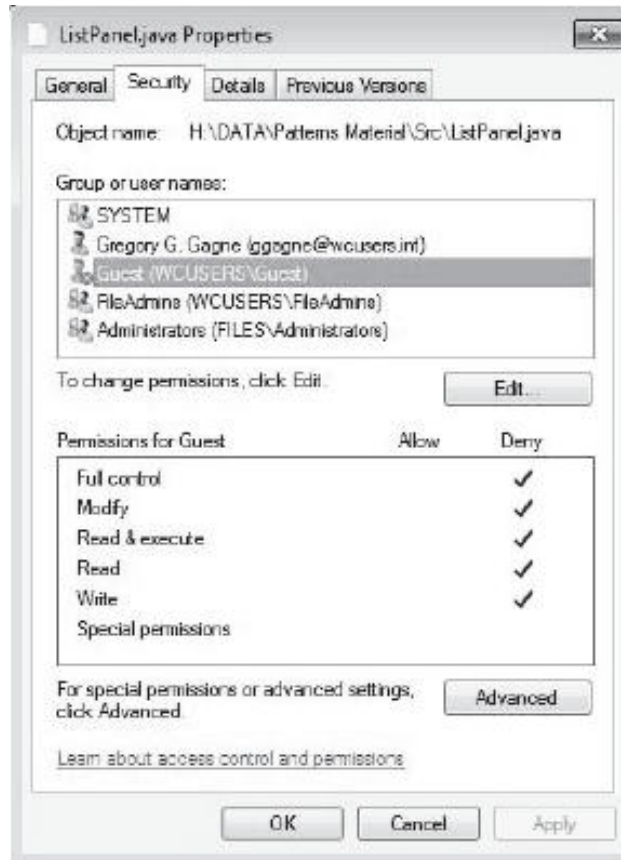
r – read permission; *w* – write permission; *x* – execute permission.

When a directory's **r** bit is set, its contents can be listed; the **x** bit permits entering (cd) the directory.

ACL Indicator

- Some UNIX variants append a **+** to the permission string to signal the presence of extended ACL entries, e.g., -rw-r--r--+.
 - Commands setfacl and getfacl manage these entries.
-

Windows ACL Example



The screenshot illustrates the Windows “Security” tab where each user/group (e.g., Guest) has explicit **Deny** checkboxes for all permission types. This GUI visualizes per-principal ACLs, mirroring the UNIX concept of an access-control list but using a graphical interface.

- Permissions are **allow/deny** for: Full control, Modify, Read & execute, Read, Write, and Special.
- The precedence rule: **Explicit deny** overrides any allow, ensuring that a user like *Guest* cannot access the file regardless of other group memberships.

This example shows how different OS families expose ACLs: Windows via a property-sheet GUI, UNIX via textual permission strings plus optional ACL extensions.

🔑 Other Protection Approaches

Password-Protected Files

- Each file can be associated with a **password** that must be presented before any operation is allowed.
- **Drawbacks:**
 - Users must remember many passwords, reducing usability.
 - If a single password protects multiple files, compromise of that password grants access to all protected files (all-or-none risk).

Directory-Level Protection

- Permissions can be attached to **directories** instead of individual files, controlling creation, deletion, and listing of the files they contain.

- Protecting a directory also protects the **existence** of its entries; listing a directory may reveal sensitive filenames, so directory-listing permission is a distinct security concern.

Handling Multiple Names (Hard Links, Graph Directories)

- When a file has several names (hard links or acyclic-graph directories), the OS must ensure that access rights are evaluated **per path**.
- A user may have permission via one path but not another; the most restrictive result is typically applied.



Summary of Protection Mechanisms

Mechanism	Granularity	Typical Use-Case	Main Advantage	Main Limitation
Owner/Group/World bits	File-level (9 bits)	Simple multi-user systems	Minimal overhead, easy to understand	Coarse; cannot express per-user exceptions
ACLs	File-level (list of users)	Fine-grained corporate policies	Flexible, supports many users/groups	Management complexity, larger metadata
Password protection	File-level	Encrypted archives, ad-hoc sharing	No OS support needed	Poor scalability, single-point failure
Directory permissions	Directory-level	Controlling whole project trees	Single point to protect many files	Does not protect individual file operations
Immutable shared files	File-level (read-only)	Distributing reference data, firmware	No consistency problems	Cannot be updated without creating a new file

These mechanisms complement the **consistency semantics** introduced earlier (session semantics, immutable files) and the **file-sharing models** (single-image vs. multiple concurrent images). Selecting an appropriate protection strategy depends on the system's scale, the sensitivity of data, and the required collaboration model.



File-System Implementation Overview

File-system implementation – the combination of on-disk structures and in-memory data that enables processes to create, open, read, write, and delete files via system calls such as **open()**, **close()**, **read()**, and **write()**.

Key on-disk structures (vary by OS)

Structure	Primary Role	Typical OS name
Boot Control Block	Holds boot-loader code and minimal OS metadata; resides in the first block of a volume.	UFS → <i>boot block</i> ; NTFS → <i>partition boot sector</i>
Volume Control Block	Describes the whole volume: total blocks, block size, free-block management pointers.	UFS → <i>superblock</i> ; NTFS → <i>master file table (MFT)</i>

Directory Structure	Maps file names to identifiers (inode numbers, MFT entries).	UFS → directory file; NTFS → entries in the MFT
File Control Block (FCB)	Stores per-file metadata: owner, timestamps, size, block pointers, ACLs.	In-memory cache of the on-disk record
Mount Table (in-memory)	Tracks each mounted volume and its associated superblock.	
Directory-cache	Holds recently accessed directory entries to avoid repeated disk reads.	
System-wide Open-File Table	Contains a copy of the FCB for every file currently open and a reference count.	
Per-process Open-File Table	Points to the system-wide entry and records the file pointer, access mode, etc.	

Open / Close Flow

1. `open(name, flags)`
 - Search the system-wide open-file table; if present, increment its reference count.
 - Otherwise, locate the name in the directory (often cached), copy its FCB into the system-wide table, and create a per-process entry.
2. `close(fd)`
 - Decrement the reference count in the system-wide table.
 - When it reaches zero, write any updated metadata back to the on-disk directory and release the entry.

UNIX calls the returned identifier a **file descriptor**; *Windows* calls it a **file handle**.



Partitions, Raw Disks, and Mounting

Concept	Description	Typical Use
Partition	A contiguous slice of a physical disk that may contain a file system or raw data.	Separate OS installations, swap space, database files
Raw Disk	Unformatted space accessed directly without a file system.	Swap partitions, database storage, RAID metadata
Mounting	Attaches a file system to a directory (UNIX) or a drive letter (Windows) so its hierarchy becomes part of the global namespace.	Makes <code>/dev/sda1</code> accessible as <code>/</code> or <code>C:</code>

Mount operation steps (UNIX style)

1. Verify the device contains a recognizable file-system format.
2. Read the superblock to obtain volume parameters.
3. Insert an entry into the in-memory **mount table** pointing to the superblock.
4. Mark the mount-point directory's inode with a **mount flag** and store a pointer to the mount table entry.

On Windows, the pointer is stored in the device structure for the corresponding drive letter.

Virtual File System (VFS) Layer

Virtual File System – an abstraction that presents a uniform interface (read, write, open, close) to user programs while delegating actual operations to the appropriate concrete file-system driver (local, network, etc.).

The VFS architecture comprises three layers:

1. **File-System Interface** – system-call entry points (read(), write(), close(), ...).
2. **VFS Layer** – defines generic objects and operation tables:
 - **inode** – abstract representation of a file (unique within a single file system).
 - **dentry** – directory entry, links a name to an inode.
 - **superblock** – per-volume information.
 - **file** – open-file instance (holds file pointer, flags).
3. **Concrete File-System Drivers** – implement the operation tables for specific types (ext4, NTFS, NFS, etc.).

When a request arrives, the VFS locates the vnode (inode) associated with the pathname, selects the appropriate driver, and invokes the driver's function via the operation table—*without* the caller needing to know the underlying file-system type.

Directory Implementation Strategies

1. Linear List

- **Structure:** Simple array of fixed-size entries, each containing a filename and a pointer (inode/FCB).
- **Operations:**
 - *Create:* Scan for a free slot, insert at the end.
 - *Delete:* Mark slot free (blank name, unused bit) or move the last entry into the freed slot.
- **Pros:** Easy to program; minimal overhead.
- **Cons:** Linear search for a name $\Rightarrow O(n)$ time; becomes a bottleneck as directory size grows.

2. Sorted Linear List

- Maintains entries in lexical order, enabling **binary search** ($\approx O(\log n)$).
- Insertion requires shifting subsequent entries, which can be expensive for large directories.

3. Hash Table

- **Hash function** maps a filename to a bucket index; each bucket holds a linked list of colliding entries.
- **Pros:** Average-case lookup near **$O(1)$** ; insertion/deletion cheap.
- **Cons:** Fixed table size; resizing forces a rehash of all entries, which can be costly.

Issue	Linear List	Sorted List	Hash Table
Search speed	$O(n)$	$O(\log n)$	$O(1)$ avg

Insertion cost	$O(1)$ (if free slot)	$O(n)$ (shifts)	$O(1)$
Deletion cost	$O(1)$ (mark)	$O(\log n) + \text{shift}$	$O(1)$
Memory overhead	Low	Low	Extra pointers + table

4. Tree Structures (e.g., B-tree)

- Not detailed in the transcript but commonly used in modern systems for scalable directories.

File-Space Allocation Methods

1. Contiguous Allocation

- **Definition:** A file occupies a consecutive run of blocks $[b, b+n-1]$.
- **Advantages:** Minimal seek time; direct indexing makes random access trivial.
- **Disadvantages:**
 - **External fragmentation** – free space becomes split into small holes.
 - Difficult to predict required size; extending a file may require moving it.

Free-space selection strategies

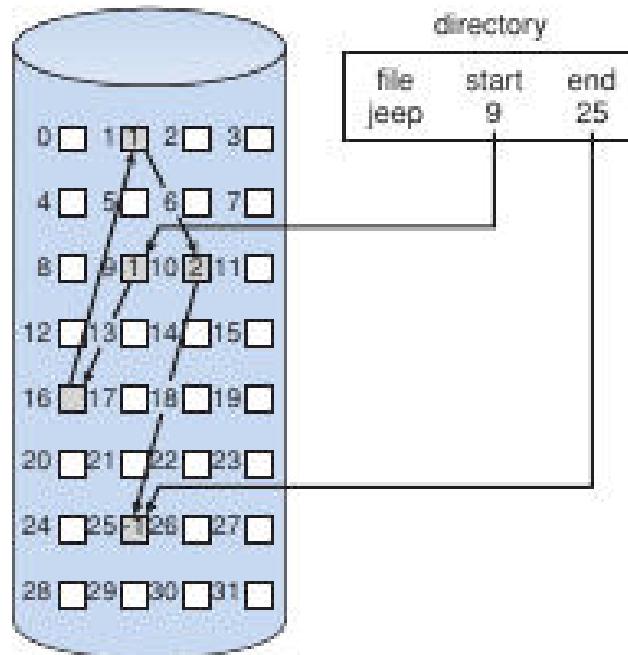
Strategy	Description	Typical behavior
First Fit	Choose the first hole large enough.	Fast, may leave many small holes.
Best Fit	Choose the smallest hole that fits.	Reduces wasted space but slower.
Worst Fit	Choose the largest hole.	Attempts to leave medium-sized holes for future allocations.

When fragmentation becomes severe, **compaction** (copying all files to a fresh disk layout) restores a single large free region, at the cost of downtime or background I/O.

2. Linked Allocation

- **Structure:** Each block stores a pointer to the next block; the directory entry points to the first block.
- **Pros:** No external fragmentation; any free block can be used.
- **Cons:**
 - Inefficient for random access – must follow the chain.
 - Each block wastes pointer space (e.g., 4 bytes per 512-byte block).

Illustration: The linked allocation of file “jeep” is shown below. The directory lists start=9, end=25, while the actual blocks used are 10, 14, 18, 22, 26, linked via pointers.



The diagram visualizes non-contiguous blocks linked together, highlighting the pointer chain that enables sequential reads despite scattered physical locations.

3. Indexed Allocation

- **Concept:** The directory (or a separate **index block**) contains an array of block numbers belonging to the file.
- **Benefits:** Direct random access without traversing a chain; pointer overhead is limited to the index block(s).
- **Variants:**
 - **Single-level index** – one block holds all pointers (suitable for modest file sizes).
 - **Multi-level (e.g., UNIX inode)** – indirect, double-indirect, triple-indirect blocks to support very large files.

4. Extent-Based Allocation

- Files are stored as a list of **extents** (contiguous runs).
- Each extent entry records **starting block** and **length**; the directory or a separate table links extents together.
- Balances the low-seek advantage of contiguous allocation with the flexibility of linked allocation.

Method	Internal fragmentation	External fragmentation	Random-access cost
Contiguous	Low (if sized correctly)	High (holes)	$O(1)$
Linked	High (pointer per block)	None	$O(n)$ for block k
Indexed	Low (only index blocks)	None	$O(1)$ with index lookup
Extent	Moderate (wasted space per extent)	Moderate (holes between extents)	$O(\log e)$ where e = number of extents

Interaction with Caching and Performance

- The **BSD UNIX** implementation achieves an **≈85% cache hit rate** by keeping most metadata (FCBs, directory entries, superblocks) in memory.
 - The **system-wide open-file table** and **per-process tables** serve as caches; repeated opens of the same file avoid extra directory scans.
 - VFS-level objects (inode, dentry) are also cached, enabling rapid resolution of pathnames across heterogeneous file-system types.
-

Summary of Allocation Trade-offs

- **Contiguous** → fastest I/O, hardest to grow files, suffers fragmentation.
- **Linked** → flexible growth, cheap space use, poor random access, pointer overhead.
- **Indexed** → good random access, minimal external fragmentation, extra index blocks.
- **Extent** → hybrid approach, widely used in modern file systems (e.g., NTFS, XFS, Veritas).

Choosing an allocation scheme depends on workload characteristics (sequential vs. random access), typical file size distribution, and the importance of minimizing seek latency versus storage overhead.

File-Allocation Methods

Linked Allocation – Drawbacks

Linked allocation stores each file block with a pointer to the next block.

- Accessing the i -th block requires traversing i pointers, causing many **disk seeks**.
- Each pointer consumes space (e.g., 4 bytes per 512-byte block → ~0.78% overhead).
- Reliability issue: a corrupted or lost pointer can break the chain, potentially linking free space into a file.
- **Doubly-linked** lists and storing file names in each block mitigate some errors but add further overhead.

Clustering

- Group several physical blocks into a **cluster** (e.g., 4 blocks) and allocate clusters instead of individual blocks.
- Benefits: fewer pointer reads, reduced head movement, and higher **throughput**.
- Cost: increased **internal fragmentation** when a cluster is only partially filled.

File-Allocation Table (FAT)

A FAT is a table placed at the start of a volume, with one entry per disk block.

- The directory entry points to the **first block** of a file.
- The FAT entry for that block contains the **next block number**; the chain ends with a special EOF marker.
- An entry of **0** indicates a free block.
- Allocation: find a 0-valued entry, link it into the chain, and update the previous EOF entry.

Performance note:

- Every block access may require reading the FAT (one seek) plus seeking to the data block itself.
- Caching the FAT dramatically reduces this overhead.

Indexed Allocation

Indexed allocation places all block pointers of a file in a single **index block**.

- The directory entry stores the **address of the index block**.
- Each entry in the index block points directly to a data block, enabling true **direct access**.
- No external fragmentation; any free block can satisfy a request.

Multilevel Index

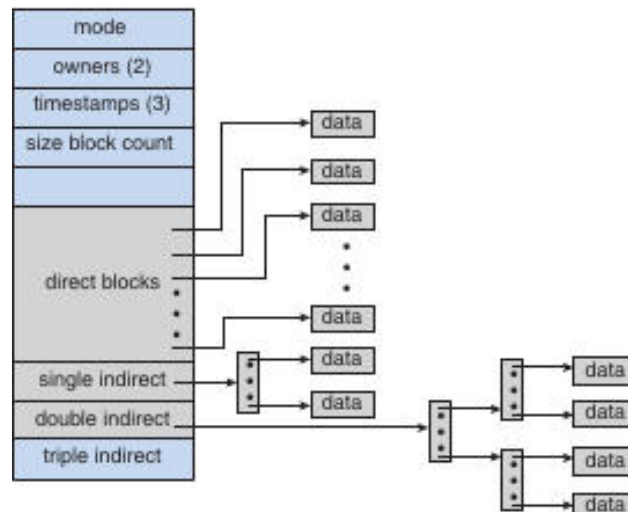
- When a file exceeds the capacity of a single index block, the last entry points to another **index block** (second level), and so on (third level if needed).
- With a 4 KB block holding 1024 four-byte pointers, two levels can address up to 4 GB of data.

Combined Scheme (UNIX inode)

- An inode contains a mixture of **direct** and **indirect** pointers:

Pointer type	Purpose
Direct (≈ 12)	Point directly to data blocks (good for small files).
Single indirect	Points to a block that contains data-block addresses.
Double indirect	Points to a block of pointers to single-indirect blocks.
Triple indirect	Points to a block of pointers to double-indirect blocks.

- Example diagram:



The image shows the inode fields (mode, owners, timestamps, size) and arrows to data blocks via direct and various levels of indirect blocks, illustrating how large files are addressed.

- Modern systems use 64- or 128-bit pointers (e.g., ZFS) to support files of many exbibytes.

Performance Implications

- **Contiguous allocation**: one seek per access; ideal for sequential workloads.
- **Linked allocation**: fine for sequential reads but poor for random access (i -th block $\rightarrow i$ seeks).
- **Indexed allocation**: constant-time block lookup; may require multiple index-block reads for multilevel schemes.

- Systems often **mix** strategies: small files use contiguous blocks; larger files switch to indexed allocation.

Free-Space Management

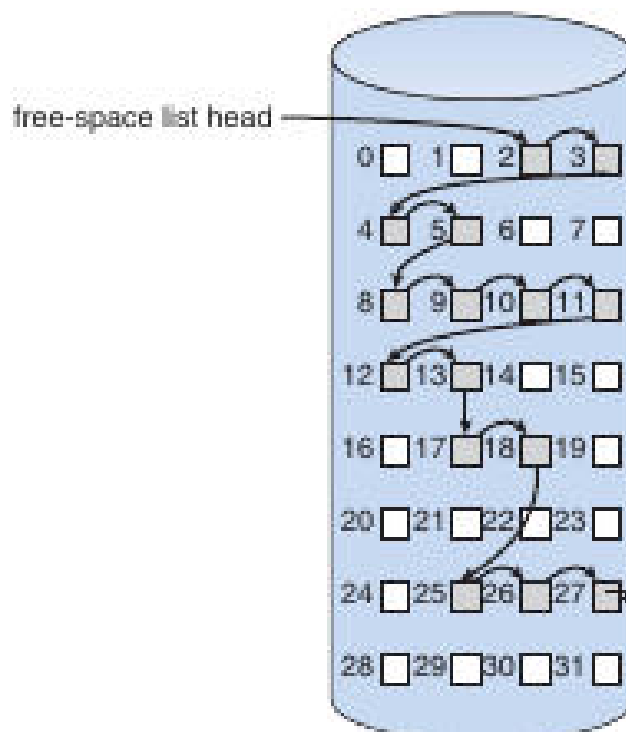
Bit Vector (Bitmap)

A bitmap uses one bit per disk block: 1 = allocated, 0 = free.

- Simple to implement; locating the first free block can be done with fast **bit-manipulation** instructions.
- Drawbacks: the bitmap must reside in memory; for very large disks (e.g., 1 TB with 4 KB blocks) the bitmap itself can be tens of megabytes.

Linked-List Free Space

- Each free block stores a pointer to the next free block; a **head pointer** is kept in a known location.
- Example traversal:



The diagram depicts a cylindrical view of disk blocks with arrows linking free blocks together, starting from the “free-space list head.”

- Inefficient for large free-space pools because every allocation requires reading multiple blocks.

Grouping (Clustered Free List)

- Store the addresses of n free blocks in a single block; the first $n-1$ entries are actual free block numbers, and the last entry points to another grouping block.
- Reduces the number of I/O operations needed to locate free space.

Counting (Extent)

- Track a **starting block address** and a **count** of consecutive free blocks.
- Entries are shorter than a full list when many contiguous free blocks exist.
- Often stored in a **balanced tree** for fast lookup, insertion, and deletion.

Space Maps (ZFS)

ZFS uses “metaslabs” and per-metaslab space maps to manage free space efficiently.

- Disk is divided into **metaslabs** (manageable chunks).
- Each metaslab has an in-memory **space map** that logs allocations and frees in a compact, log-structured format.
- When the metaslab is committed, the space map is written as a single record, minimizing scattered bitmap updates.

Efficiency & Performance Considerations

Allocation Method Choice

- **Sequential-access** workloads benefit from **linked** or **contiguous** allocation.
- **Random-access** workloads require **indexed** allocation (or FAT with caching).
- Some systems require the file type to be declared at creation (sequential vs. direct) and may convert by copying data to a new file.

Caching Strategies

- **Buffer cache**: a memory area that holds recently read disk sectors.
- **Page cache** (unified with virtual memory): caches file data as virtual pages, allowing both `read()/write()` and memory-mapped I/O to share the same cache.

Unified buffer cache avoids “double caching,” where data would be stored both in the buffer cache and the page cache, saving memory and CPU cycles.

- Modern OSes (Solaris, Linux, Windows) employ a **unified cache**, so memory-mapped files and regular I/O calls use the same underlying pages.

Impact of Pointer Size

- 32-bit pointers limit file size to ~4 GB.
- 64-bit pointers expand the limit to exabytes, but increase metadata size and memory pressure.
- ZFS adopts **128-bit pointers**, future-proofing for massive storage capacities.

Evolution of System Structures

- Early systems used **fixed-size tables** (process table, open-file table); once full, no more processes/files could be created.
- Later kernels allocate these structures **dynamically**, removing artificial limits at the cost of more complex management.

Summary Tables

Allocation Method	Direct-Access Support	External Fragmentation	Internal Fragmentation	Typical Use
Contiguous	✓ (simple calculation)	✗ (fixed start)	Low (if file size matches)	Small, sequential files
Linked	✗ (requires traversal)	✓ (any free block)	Low (no waste per block)	Purely sequential streams
FAT	✓ (via table)	✓ (any free block)	Low (table overhead)	DOS/early Windows
Indexed (single)	✓ (index lookup)	✓ (any free block)	Moderate (index block overhead)	General-purpose FS
Multilevel Index / Inode	✓ (scales)	✓	Higher (multiple indirect blocks)	Modern UNIX, Linux
Clustered (grouped)	✓ (cluster as unit)	✓	Higher (partial cluster waste)	BSD, ext* filesystems
Free-Space Scheme		Memory Overhead	I/O Overhead	Scalability
Bit vector		Proportional to disk size	Low (single scan)	Excellent for moderate disks
Linked list		One pointer per free block	High (traversal)	Poor for large free pools
Grouping		One block per n free blocks	Moderate	Good when many free blocks
Counting (extent)		One entry per contiguous run	Low (direct allocation)	Very good for fragmented disks
ZFS space map		In-memory per metaslab	Very low (log-structured)	Excellent for huge volumes

These notes integrate allocation strategies, free-space management techniques, and performance-optimizing mechanisms, building on earlier concepts of TLB reach, paging, and disk I/O.

Page Scanner & Priority Paging

Page scanner – a background daemon that reclaims pages from a process's working set (not from the page cache) when free memory becomes scarce.

- Early Solaris 2.6/7 used a **fixed limit** on the number of pages a priority-process could retain, preventing it from evicting pages that the file-system cache needed.
- Solaris 9/10 replaced the fixed-limit scheme with **dynamic priority paging**, which tries to *maximise memory utilisation* while *minimising thrashing* by adjusting reclamation rates according to overall system load.

I/O Modes: Synchronous vs. Asynchronous

Synchronous I/O – the calling process blocks until the data have physically reached the disk.

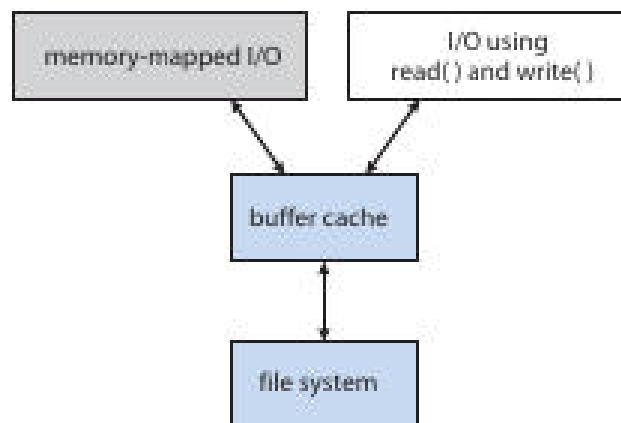
Asynchronous I/O – the kernel copies data into the buffer cache and returns to the caller immediately; the actual disk write occurs later.

- Most regular file writes are **asynchronous**, improving throughput because the CPU can continue computation while the disk controller works.
- **Metadata writes** (e.g., directory updates, inode changes) are often **synchronous** to guarantee atomicity for transactions.
- Many systems expose a flag in the `open()` call (e.g., `O_SYNC`) that forces all subsequent writes on that descriptor to be synchronous, useful for database commit logging or other durability-critical workloads.

I/O mode	Caller behaviour	Typical latency impact	Typical use-case
Synchronous	Blocks until completion	High (wait for seek + rotational latency)	Metadata updates, transactional logs
Asynchronous	Returns immediately, kernel buffers data	Low (CPU continues)	Bulk data writes, streaming

Page-Cache Optimisation Strategies

The kernel's page cache sits between **memory-mapped I/O** / `read()` / `write()` and the underlying **file system**.



The diagram shows how both memory-mapped accesses and explicit read/write system calls funnel through the buffer cache before reaching the file system, highlighting the central role of the page cache in I/O performance.

Sequential-write optimisation

- Pure **LRU** replacement is sub-optimal for a file that is being written sequentially; the most-recently-used page will never be needed again after the write completes.
- **Free-behind** discards a page from the buffer cache as soon as it has been written, freeing space for future pages.

Read-ahead (prefetch)

- When a page is requested, the driver can also fetch **N** subsequent pages (e.g., the next 2–4 blocks) into the cache, because they are likely to be accessed soon.
- Even with a hardware track cache, read-ahead remains valuable on rotating disks because issuing many small transfers incurs extra seek and latency overhead.

Disk-driver queue sorting

- After pages are placed in the buffer cache, the disk driver **reorders** the pending write queue based on cylinder numbers (or more generally on seek distance) to minimise head movement.
- This *write-behind* optimisation is most effective when writes are **asynchronous**; the driver can batch several pending buffers into a single long write.

Recovery & Consistency Checking

A crash can leave on-disk structures (directories, free-block maps, inode tables) in an inconsistent state.

- **Metadata-status bit**: set at the start of any metadata-changing operation; cleared only after the operation completes. If the bit remains set after a reboot, a **consistency checker** (fsck) is invoked.
- fsck walks the entire directory hierarchy, verifies that every allocated block is referenced exactly once, and that free-block lists are correct.

Limitations of pure post-crash checking:

- May require **hours** on large volumes.
- Some inconsistencies are **irrecoverable** (e.g., lost directory entries).
- Human intervention is often needed to decide how to resolve ambiguous cases.

Log-Based (Journaling) Recovery

Write-ahead log (journal) – a sequential area where all metadata updates are recorded before being applied to the main file-system structures.

- A **transaction** groups a set of related updates (e.g., file creation, inode allocation).
- The transaction is first written to the log; only after the log write is confirmed does the kernel apply the changes to the on-disk structures.
- The log is typically a **circular buffer**: when the end is reached, it wraps to the beginning, overwriting the oldest, already-committed entries.

Recovery steps after a crash

1. Scan the journal from the last known **checkpoint**.
2. **Replay** any transaction that was fully recorded but not yet reflected in the file system.
3. **Undo** any partially recorded transaction (if the system tracks undo information) or simply discard it.

Benefits:

- Eliminates the need for a full fsck run in the common case.
- **Sequential I/O** to the journal is far faster than random writes to disparate metadata locations.
- Metadata-heavy operations (file creation, deletion, permission changes) become **orders of magnitude faster**.

Copy-On-Write File Systems: WAFL & ZFS

- **WAFL** (Write Anywhere File Layout) writes new data and metadata to fresh blocks; the old blocks remain untouched until the transaction commits. This enables **instant snapshots** because the old block pointers can be preserved.
- **ZFS** extends the COW idea and adds **checksumming** for every block (data + metadata). On read, the checksum is verified; corrupted blocks are automatically repaired from redundant copies (mirrored or

RAID-Z groups).

- Both systems avoid the classic “overwrite-then-crash” problem, so a separate consistency checker is unnecessary; the on-disk state is always **self-consistent**.

Backup & Restore Strategies

Common backup schedules

Day	Action	Type
1	Copy all files to backup medium	Full backup
2-N	Copy only files whose modification time is later than the previous backup	Incremental backup
N+1	Repeat the full backup and start a new cycle	Cycle reset

- **Full + Incremental** approach limits the amount of media needed while still allowing a complete restore: start with the most recent full backup, then replay each incremental in order.
- **Media wear**: when reusing tapes or disks, track the number of write cycles to avoid premature failure.
- **Permanent off-site copy** (e.g., fire-proof vault) protects against catastrophic loss of both primary storage and the local backup set.

Restoring a single file

- Because each incremental records only changed files, a file deleted accidentally can be recovered from the **most recent incremental** that still contains it (or from the preceding full backup).

Network File System (NFS) Overview

NFS – a client-server protocol that lets remote files appear in a local directory hierarchy as if they were on a local disk.

- Mounting a remote export replaces a local directory subtree with the remote file system, providing **transparent access**.
- The protocol is **stateless**: the server does not retain per-client open-file tables, making crash recovery trivial.

The NFS Mount Protocol

- Each server maintains an **export list** (e.g., /etc/dfs/dfstab on Solaris) that names the local file systems available for mounting and the client hosts permitted to use them.
- A client issues a **mount RPC** containing the server name and the remote directory path.
- The server responds with a **file handle**: a compact identifier consisting of the file-system ID and the inode number of the root of the exported subtree.
- The client stores the handle locally; subsequent file operations use it to address the remote resource without further mount negotiation.

NFS Protocol Details

- The protocol is a set of **RPC procedures** (e.g., READ, WRITE, GETATTR, LOOKUP, REaddir, LINK, REMOVE).
- **Statelessness** ⇒ every request carries the full file handle and the byte offset; the server does not keep client state between calls.
- **Idempotence** is achieved by attaching a **sequence number** to each request; the server can detect and discard duplicates caused by retransmission.
- Writes are limited to **8 KB** per RPC (on UDP); larger write() calls are split into multiple RPCs, which can interleave with other clients' writes.
- Because NFS itself provides no locking, a separate **lock manager** (often implemented as part of the OS) is required for coordinated access to shared files.

Performance considerations

- **Synchronous writes** (default) wait for the server to flush data to stable storage; this can be mitigated by using **non-volatile cache** (battery-backed RAM) on the server, which acknowledges the write once the data reach the cache.
- **Read-ahead** on the client side can reduce round-trip latency for sequential reads, while **attribute caching** (inode information) avoids repeated GETATTR RPCs.

Path-Name Translation in NFS

- A client parses a pathname (e.g., /usr/local/dir1/file.txt) into components and performs a **lookup RPC** for each component until a mount point is reached.
- After the mount point, the client switches to the remote server's namespace, issuing further lookups there.
- To avoid the overhead of repeated component-wise lookups, the client maintains a **directory-name cache** that stores vnode information for recently accessed remote directories.
- **Cascading mounts** (mounting a remote export on top of another remote mount) cause the client to traverse multiple servers; the cache must be keyed by the combination of server and directory to stay correct.

Remote Operations & Caching

- Two primary caches exist on the client:

Cache type	Stores	Validation rule
Attribute cache	Inode metadata (size, timestamps, permissions)	Revalidated when a server-supplied attribute differs from the cached copy.
Block cache	File data blocks (pages)	Used only if the corresponding attributes are up-to-date; otherwise the block is discarded and refetched.

- When a file is opened, the client may **prefetch** both attributes and the first data block, reducing the latency of the first read.
- Writes follow the **write-behind** pattern: modified blocks are placed in the block cache and sent to the server asynchronously; the client assumes the server will eventually commit them to stable storage.

These notes extend the earlier discussions of **page-cache behaviour**, **TLB reach**, and **disk-scheduling** by showing how modern OSes tune memory reclamation, optimise sequential I/O, and maintain file-system consistency both on-disk (journaling, COW) and across the network (NFS).

NFS Consistency Semantics

Definition: *Delayed-write* – a technique where modified blocks are kept in the client's buffer cache and not written to the server until later.

- Attributes are **discarded after 60 reads** when read-ahead and delayed-write are active.
- A client **does not free delayed-write blocks** until the server has confirmed that the data are on disk.
- Even when a file is opened **concurrently in conflicting modes**, delayed-write remains in effect, breaking the **UNIX semantics** described in §10.5.3.1.
- **Visibility delays:**
 - New files created on one machine become visible elsewhere after **≈30 s**.
 - Writes to a file may **or may not be visible** to other sites that already have the file open; those sites only see changes that have already been flushed to the server.
- Consequently, **NFS does not provide strict UNIX semantics nor the session semantics of Andrew File System** (§10.5.3.2).
- Tuning NFS for performance therefore **obscures its consistency guarantees**, making it hard to characterize exactly when a write becomes visible to other clients.

WAFL File System Overview

Definition: *Write-Anywhere File Layout (WAFL)* – a file-system design that writes new data to any free block rather than overwriting existing blocks, enabling efficient random-write performance and snapshot capability.

- Developed by **Network Appliance** for its dedicated network-file servers (NFS, CIFS, FTP, HTTP).
- **Key motivations:**
 - Random-write workloads dominate on file servers serving many NFS/CIFS clients.
 - NFS/CIFS cache reads heavily, so **writes are the primary performance bottleneck**.
- WAFL runs on **hardware with a stable-storage NVRAM cache** in front of the disks, allowing writes to be acknowledged as soon as they reach the NVRAM.
- **Design principles:**
 - Simplicity of use (ease of administration).
 - **Snapshot** functionality built-in – creates read-only views of the file system at a point in time with minimal extra storage.

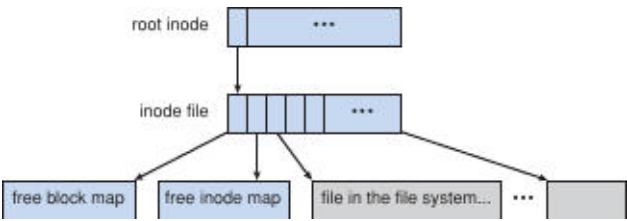
WAFL Structure & Metadata

- Block-based, inode-driven, similar to the Berkeley Fast File System.
- Each inode contains **16 direct pointers** (or indirect pointers) to data blocks.
- All **metadata files** are regular data blocks, not fixed-location structures:

File	Purpose
inode file	Stores every inode in one logical file.

free-block map	Bitmap indicating which blocks are free.
free-inode map	Bitmap indicating which inodes are free.

- The **root inode** is the entry point to the entire WAFL tree.



The diagram shows the root inode branching to the inode file, which in turn links to three key components: the free-block map, the free-inode map, and the regular user files (grey). This visual ties the logical hierarchy to the physical block layout used by WAFL.

Snapshot Mechanism 📸

- **Copy-on-Write (COW) of the root inode:**
 1. When a snapshot is requested, WAFL **copies the root inode** (not the data blocks).
 2. Subsequent metadata and data updates are written to **new blocks**, leaving the original blocks untouched.
 3. The snapshot's root inode continues to point to the **old, unmodified blocks**, providing an exact view of the file system at snapshot time.
- **Space efficiency:** Only blocks that change **after** the snapshot consume additional space.
- Example progression (Figure 11.17 in the source):

Stage	Description
(a) Before snapshot	Root inode → blocks A, B, C.
(b) After snapshot, before new writes	Snapshot root inode points to A, B, C; live root inode still points to A, B, C.
(c) After a write to block C	Live root inode now points to A, B, D (new block); snapshot root inode still points to A, B, C.

- **Bitmap per-snapshot:** The free-block bitmap stores **more than one bit per block**—one per snapshot that still references the block. When the last snapshot referencing a block is deleted, the bitmap entry becomes zero and the block can be reclaimed.

Clone Feature & Read-Write Snapshots 🧬

- WAFL supports **read-write snapshots (clones)**:
 - A clone starts as a **read-only snapshot**, then receives its own writes, which are allocated to new blocks just like normal writes.
 - The original snapshot remains unchanged, preserving the historic view.
- **Promotion:** A clone can be promoted to become the primary file system; this discards the old primary's blocks and adopts the clone's block set.
- **Use cases:**
 - Testing upgrades without affecting production data.
 - Rapidly provisioning development environments from a known good state.

Replication via Snapshots

1. **Initial replication:** Take a snapshot of the source WAFL volume and copy it to a remote system.
 2. **Incremental updates:** Subsequent snapshots capture only the blocks that changed since the previous snapshot. The remote system receives just those new blocks and updates its pointers accordingly.
 3. The remote copy thus **mirrors the source** as of the latest snapshot, providing near-real-time disaster-recovery capability.
- The same snapshot-based approach is used by **ZFS**, which also offers snapshots, clones, and replication.

File-System Implementation Recap

Concept	Core Idea
Permanent storage	Files reside on secondary storage (usually magnetic disks); partitions allow multiple logical file systems on one device.
Mounting	Attaches a file system to a directory (UNIX) or drive letter (Windows), making its hierarchy part of the global namespace.
VFS layer	Provides a uniform API (open, read, write, close) while delegating to concrete file-system drivers (e.g., UFS, NTFS, ZFS, WAFL).
Allocation methods	Contiguous , linked , indexed , extent-based , and clustered allocations each trade off seek cost, fragmentation, and metadata overhead.
Free-space management	Bitmaps, linked free lists, grouping, counting, and FAT-style tables; WAFL's per-snapshot bitmap extends the bitmap idea to support COW snapshots.
Directory structures	Linear lists, sorted lists, hash tables, and tree-based directories; hash tables give $O(1)$ lookup on average but require crash-consistent handling.
Caching	Unified buffer/page cache stores both file data and metadata; read-ahead and write-behind improve sequential performance, while free-behind discards pages after write-back to free cache space.
Consistency mechanisms	<i>Log-structured</i> or <i>journalled</i> approaches (e.g., WAFL's COW, ZFS's intent log) ensure that on-disk structures are either fully applied or not applied after a crash, avoiding full fsck scans.
Backup & restore	Full + incremental schemes; WAFL snapshots act as fast, space-efficient "on-disk backups," and replication copies snapshots to remote sites for disaster recovery.

Allocation Method Comparison

Method	Direct-access cost	External fragmentation	Internal fragmentation	Typical use case
--------	--------------------	------------------------	------------------------	------------------

Contiguous	1 I/O per block	High (holes)	Low (if sized correctly)	Small files, sequential workloads
Linked	n I/Os for the n -th block	None	Low (pointer per block)	Purely sequential writes
Indexed (single-level)	2 I/Os (index + data)	None	Moderate (index block overhead)	Random access, moderate file size
Extent-based	$1 + e$ I/Os (where e = number of extents)	Reduced vs. contiguous	Low (large extents)	Large files with occasional growth
Clustered indexed	$1 + c$ I/Os (cluster size c)	None	Low (fewer index entries)	High-throughput servers (e.g., WAFL)

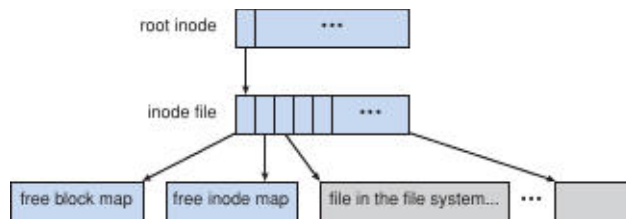
Free-Space Management Comparison

Scheme	Metadata size	Allocation speed	Reclamation cost
Bit vector	1 bit/block	Fast (find first 0)	Simple (clear bit)
Linked list	1 pointer/block	Slow (traverse)	Easy (unlink)
Grouping	Pointer to a block of free block numbers	Fast (bulk allocation)	Moderate (manage groups)
Counting (extent bitmap)	Counter per block (multiple bits)	Fast for large runs	Requires scan of counters
WAFL per-snapshot bitmap	One bit per block per active snapshot	Fast for COW writes	Blocks reclaimed when all snapshots release them

Integration with Earlier Topics

- The **delayed-write behavior of NFS** mirrors the **asynchronous I/O mode** described in the I/O chapter: writes return before data reach stable storage, affecting consistency semantics.
- **WAFL's copy-on-write snapshots** rely on the same **TLB reach and page-cache concepts** discussed earlier—metadata updates are cached in memory, and only when the NVRAM cache flushes are they persisted, preserving a consistent view for readers.
- **Replication via incremental snapshots** reduces the amount of data that must traverse the network, analogous to the **page-replacement strategies** that aim to keep hot pages in memory and minimize I/O.

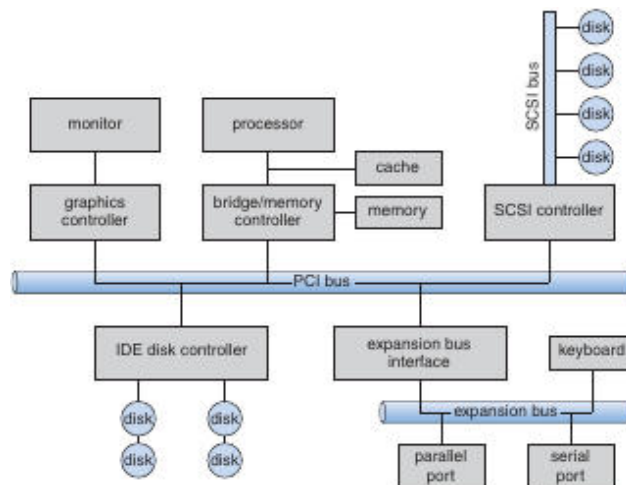
Visual Summary of WAFL Snapshot Flow



The image illustrates the root inode pointing to the inode file, which in turn references the free-block map, free-inode map, and user data. When a snapshot is taken, the root inode is duplicated; subsequent writes allocate new blocks, leaving the snapshot's block set unchanged. This COW mechanism enables efficient, space-saving snapshots.

I/O Hardware Overview

The computer's internal architecture connects the CPU, memory, cache, graphics controller, and various I/O buses.



The diagram shows the processor linked to the memory-controller/bridge, a graphics controller, a PCI bus, and peripheral controllers (SCSI, IDE, keyboard, ports). This layout illustrates the physical pathways used for I/O transfers.

I/O Controllers & Buses

- **SCSI controller** – usually a separate host-adaptor board that plugs into the PCI bus.
 - Contains its own **processor, microcode, and private memory** to interpret SCSI protocol messages.
- **Disk controller** – mounted on the drive's circuit board (SCSI or SATA).
 - Handles **bad-sector mapping, prefetching, buffering, and caching** in firmware.

These controllers act as the *intermediate* between the CPU's generic I/O requests and the device-specific command sets.

Communication Between CPU and Devices

Method	How it works
I/O instructions	The CPU writes/reads specific I/O-port addresses; the bus lines select the target device register.
Memory-mapped I/O	Device registers are mapped into the processor's physical address space; normal load/store instructions access them.

Definition: *Memory-mapped I/O* – device-control registers appear as ordinary memory locations, allowing the CPU to use standard data-transfer instructions.

PC I/O Port Map (partial)

Port range (hex)	Device
000–00F	DMA controller
020–021	Interrupt controller
040–043	Timer
200–20F	Game controller
2F8–2FF	Serial port (secondary)
320–32F	Hard-disk controller
378–37F	Parallel port
3D0–3DF	Graphics controller
3F0–3F7	Diskette-drive controller
3F8–3FF	Serial port (primary)

Memory-Mapped I/O (Graphics Example)

- **Pros**
 - Writing directly to the screen buffer is **much faster** than issuing millions of I/O instructions.
 - Simplifies code: a single pointer to the video memory suffices.
- **Cons**
 - Accidental writes to the mapped region can corrupt the display or cause a system crash.
 - Requires **protected memory** to prevent ordinary applications from reaching the video registers.

I/O Port Register Set

Typical I/O ports expose four registers:

1. **Status register** – bits indicate device state (busy, data ready, error, etc.).

2. **Data-in register** – host reads incoming data here.
3. **Data-out register** – host writes outgoing data here.
4. **Control register** – host writes commands or mode settings (e.g., full-duplex vs. half-duplex, parity enable, word length).

Note: Data registers are usually 1–4 bytes wide. Many controllers add a **FIFO** buffer to hold several bytes, reducing the need for the CPU to service each byte individually.

Polling & Handshaking

A simple *producer–consumer* handshake uses two status bits:

- **Busy bit** (in the status register) – set by the controller while it is processing a command.
- **Command-ready bit** (in the control register) – set by the host when a new command is available.

Handshaking sequence (byte-wise write example)

1. Host **polls** the busy bit until it is clear.
2. Host writes the command code to the **command-ready** bit.
3. Host writes the data byte to **data-out**.
4. Host sets the **command-ready** bit to indicate the command is ready.
5. Controller sees the command-ready bit, reads **data-out**, clears the busy bit when finished, and may clear the command-ready bit.
6. Host repeats the loop for the next byte.

When the device can generate data faster than the CPU polls, the CPU may waste cycles in a *busy-wait* loop. In such cases, **interrupts** are preferred.

Interrupts

Basic Mechanism

1. A device asserts the **interrupt-request (IRQ)** line.
2. After the current instruction, the CPU saves its state and jumps to a fixed **interrupt-handler address**.
3. The handler determines the source, services the device, clears the IRQ, restores the saved state, and returns.

Types of IRQs

IRQ line	Characteristics
Non-maskable interrupt (NMI)	Cannot be disabled; used for critical errors (e.g., memory parity).
Maskable interrupt	Can be disabled during critical sections; used by most device controllers.

Interrupt Vector & Chaining

- The **interrupt vector** is a table of entry points (addresses) for each possible IRQ.
- When the number of devices exceeds the vector size, **interrupt chaining** links multiple handlers to a single vector entry, invoking them sequentially until one claims the interrupt.

Intel Pentium Event-Vector Table (excerpt)

Vector	Description
0	Divide error
1	Debug exception
...	...
14	Page fault
15	Reserved (do not use)
16	Floating-point error
17–31	Reserved
32–255	Maskable device interrupts

Uses of Interrupts

- **Device I/O** – signal completion of a read/write or availability of new input.
- **Page-fault handling** – a missing page raises an interrupt that invokes the kernel's page-fault routine.
- **System calls** – user programs execute a special **trap** instruction (software interrupt) to request kernel services.
- **Kernel threads** – modern kernels (e.g., Solaris) implement interrupt handlers as high-priority threads, allowing multithreaded scheduling and priority-based preemption.

Direct Memory Access (DMA)

- **Purpose:** Offload bulk data transfers (e.g., disk I/O) from the CPU.
- The CPU writes a **DMA command block** to the DMA controller, specifying source address, destination address, and byte count.
- The DMA controller takes control of the memory bus, moving data directly between the device and main memory (or between two memory-mapped devices).

Handshaking Signals

Signal	Direction	Meaning
DMA-request	Device → DMA controller	Device has data ready or needs a buffer.
DMA-acknowledge	DMA controller → Device	Controller is ready; transfer proceeds.

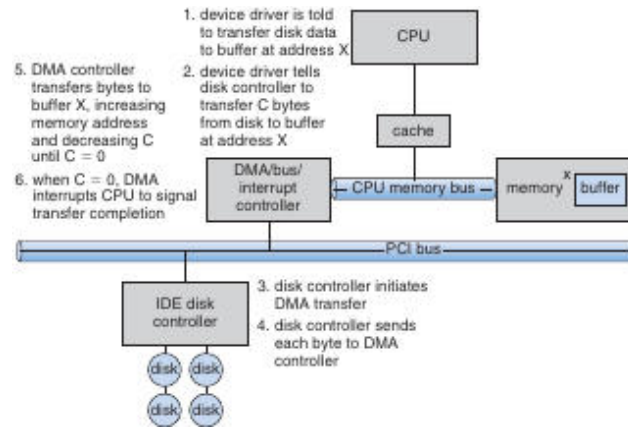
During a transfer, the CPU may be temporarily **stolen** from the bus (cycle stealing). Although this can slow CPU execution, overall system throughput improves because the CPU can perform other work while the DMA engine handles the bulk transfer.

Physical vs. Virtual DMA

- **Physical DMA** uses raw physical addresses.
- **Direct Virtual Memory Access (DVMA)** uses virtual addresses that the hardware translates to physical addresses, allowing device-to-device transfers without intermediate copies.

Security Considerations

- Unrestricted access to DMA would let a malicious process overwrite arbitrary memory. Modern kernels expose only privileged APIs that set up DMA descriptors, preventing direct user-mode manipulation.



The diagram shows a six-step DMA transfer: the device driver programs the DMA controller, the controller seizes the bus, moves data between the disk and a memory buffer, then signals completion with an interrupt.

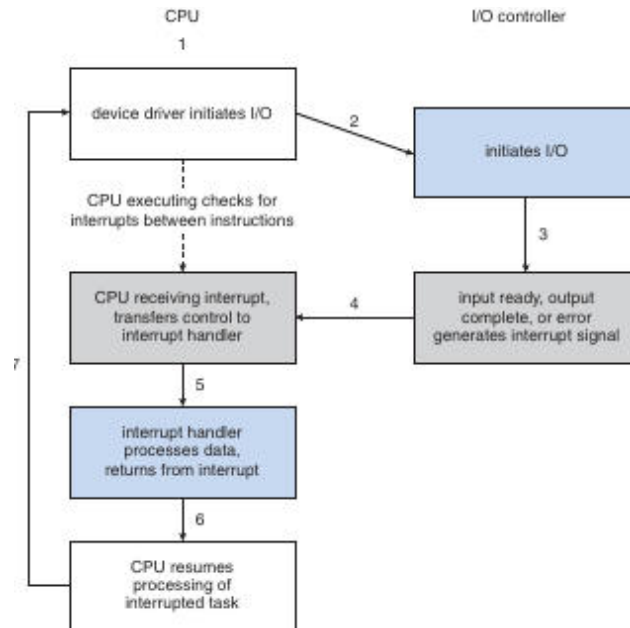
Summary of I/O Hardware Concepts

- **Bus** – shared communication pathway connecting CPU, memory, and controllers.
- **Controller** – dedicated hardware (often on a separate board) that interprets device-specific protocols.
- **I/O ports & registers** – status, data-in, data-out, control; sometimes augmented with FIFO buffers.
- **Polling** – CPU repeatedly reads status bits; simple but can waste cycles.
- **Interrupts** – hardware-driven notifications; support maskable vs. non-maskable, vectors, chaining, and priority levels.
- **DMA** – hardware engine that transfers data without CPU intervention; uses command blocks and handshaking signals; may employ cycle stealing or virtual address translation.

These mechanisms form the foundation for the higher-level **application I/O interface** covered next.

Application I/O Interface

Operating systems hide the diversity of physical devices behind **standardized interfaces**.



The flowchart illustrates the layered structure: applications → system calls → I/O subsystem → device drivers → hardware controllers. Device drivers encapsulate hardware specifics while exposing a uniform API to the kernel.

- **Abstraction** – defines a generic set of operations (e.g., read, write, seek).
- **Encapsulation** – device-specific logic resides in *device drivers* within the kernel.
- **Layering** – the kernel's I/O subsystem routes requests from system calls to the appropriate driver.

Device Classes

Class	Access pattern	Typical examples
Block device	Transfers fixed-size blocks; random-access possible	Hard disks, SSDs, CD-ROMs
Character (stream) device	Byte-oriented, often sequential	Keyboard, mouse, serial ports
Memory-mapped device	Device registers appear in the address space	Video RAM, certain network adapters
Network socket	Message-oriented or stream-oriented communication over a network	TCP/UDP sockets

Common Characteristics (from Figure 12.7)

Property	Possible values
Access method	Sequential vs. random
Synchronization	Synchronous vs. asynchronous
Sharing	Sharable vs. dedicated
Speed	Bytes/s to gigabytes/s

Direction	Read-only, write-only, read-write
-----------	-----------------------------------

Block Device Interface

- Core system calls: `read()`, `write()`, `seek()`.
- **Raw (unbuffered) access** – applications can disable OS buffering and locking to manage their own caches. In UNIX this is often invoked via the `O_DIRECT` flag.
- **Memory-mapped I/O** – the kernel can map a file's contents into the process's address space (`mmap()`), allowing the program to treat disk data as an array of bytes. This leverages the same demand-paging mechanisms used for virtual memory, providing high efficiency.

Definition: *Memory-mapped file* – a file whose contents are mapped into a process's virtual address space, enabling direct byte-level access without explicit read/write system calls.

Character Device Interface

- Basic calls: `read()` and `write()` operate on a **byte stream**.
- Libraries often provide higher-level functions such as `getchar()` / `putchar()` for keyboards or terminals.
- `ioctl()` (I/O control) is a generic system call that passes device-specific commands:

```
int ioctl(int fd, unsigned long request, void *argp);
```

- `fd` – file descriptor referring to the device.
- `request` – integer selecting the driver-specific command.
- `argp` – pointer to an arbitrary data structure for command arguments.

`ioctl()` lets applications invoke functions that are not covered by the standard read/write API (e.g., setting baud rate on a serial port or obtaining the size of a terminal window).

Linear (Character) I/O Devices

Definition: **Character devices** provide a linear stream of bytes, one at a time, often with minimal buffering.

- Typical examples: **keyboards**, **mice**, **modems**, **printers**, and **audio output**.
- Input handling: when a user types a backspace, the driver removes the preceding character from the input stream before passing the data to the application.
- Output handling: devices such as printers naturally consume a **sequential byte stream**, making the line-at-a-time model convenient.

Key point: Because the timing of data arrival cannot be predicted by the application, the OS must buffer the stream and present a simple read/write interface to the program.

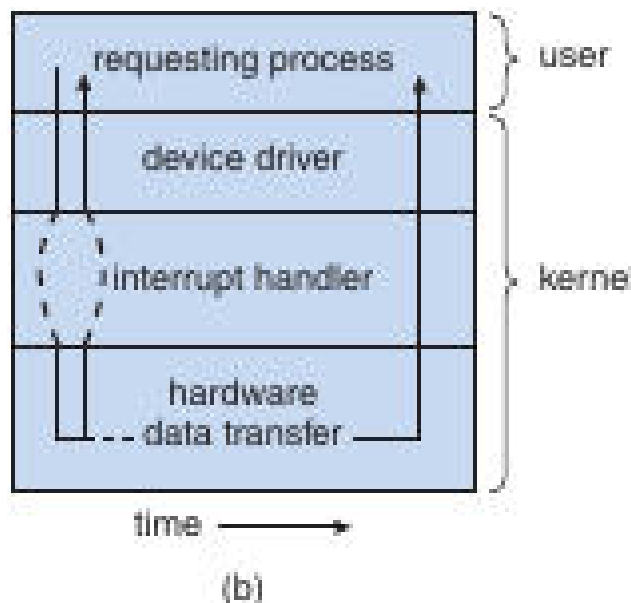
Network Devices & Socket Interface

Definition: **Sockets** are an abstraction that lets a process create an endpoint for communication, independent of the underlying hardware or protocol.

- System calls: `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()`, `recv()`.
- **`select()`**:
 - Returns information about which sockets have data ready to read or space available for writing.
 - Eliminates busy-waiting loops; a timeout value of 0 makes `select()` a non-blocking poll.

Analogy: A wall socket supplies electricity to any appliance; similarly, a network socket supplies a uniform I/O interface to any network protocol.

Illustration of Interrupt-Driven I/O



The diagram shows the path from a hardware interrupt (bottom) up through the device driver and interrupt handler, finally reaching the requesting user process (top). This layered flow demonstrates how asynchronous device events are turned into synchronous system-call completions.

Clocks and Timers

Definition: **Timers** provide three basic services:

1. **Current time** – query the system clock.
 2. **Elapsed time** – measure intervals.
 3. **Set a future timer** – generate an interrupt at a specified moment.
- **Programmable interval timer (PIT):** can be programmed for a one-shot or periodic interrupt.
 - **Uses in the OS:**
 - Scheduler time-slice preemption.
 - Periodic flushing of disk buffers.

- Network time-outs for congested or failed operations.

Virtual Clocks

- The kernel maintains a **sorted list of pending timer requests** (both kernel-generated and user-generated).
- When the hardware timer fires, the kernel services the **earliest-time-first** request and re-programs the hardware for the next event.

Resolution Considerations

Source	Typical resolution	Remarks
PIT (ticks)	18–60 Hz ($\approx$ 16-55 ms)	Coarse; sufficient for scheduling but not for high-precision profiling.
High-frequency counter	Microseconds to nanoseconds	Provides accurate interval measurement; does not generate interrupts itself.

Nonblocking vs. Asynchronous I/O

- **Blocking I/O**: the calling thread is placed on a wait queue until the operation completes. Simple for the programmer but wastes CPU time.
- **Nonblocking I/O**: the system call returns immediately with the number of bytes actually transferred (may be zero). The application must poll or use `select()/poll()` to discover readiness.
- **Asynchronous I/O**: the call returns instantly; completion is reported later via a **signal**, **callback**, or **completion event**. The kernel may copy the full request to a private buffer and perform the transfer in the background.

Aspect	Blocking	Nonblocking	Asynchronous
Caller behavior	Sleeps until done	Returns instantly	Returns instantly
How completion is reported	Implicit (resume)	Explicit (poll)	Explicit (signal / callback)
Typical use case	Simple utilities	Event-driven servers	High-throughput media pipelines

Threading tip: A multithreaded program can let one thread perform blocking I/O while others continue computation, achieving overlap without nonblocking calls.

Vectored (Scatter-Gather) I/O

Definition: **Vectored I/O** allows a single system call to read from or write to multiple, non-contiguous buffers.

- System calls: `readv()`, `writev()`.
- **Scatter read**: data from a single source is placed into several buffers (useful for protocol headers + payload).
- **Gather write**: data from several buffers is transmitted as a single continuous stream.

Advantages

- Reduces the number of system-call transitions (lower overhead).
- Enables **atomic** transfer of all buffers, preventing interleaving with other threads.
- Facilitates **zero-copy** networking where packet fragments are assembled directly from user buffers.

Kernel I/O Subsystem Overview

Subsystem	Primary role
Scheduling	Orders pending I/O requests to minimize seek time and ensure fairness.
Buffering	Holds data temporarily to bridge speed mismatches between producer and consumer.
Caching	Keeps copies of frequently accessed disk blocks in fast memory.
Spooling	Queues output for devices that can handle only one job at a time (e.g., printers).
Device reservation	Guarantees exclusive access when required (e.g., tape drives).
Error handling	Detects, retries, and reports hardware failures.
Protection	Enforces privileged access to I/O instructions and memory-mapped registers.
Power management	Powers down idle devices and CPUs to save energy.

1 I/O Scheduling

- The OS maintains a **request queue per device**.
- Example ordering (FCFS → SCAN → C-SCAN) reduces average seek distance, as illustrated by the earlier disk-scheduling discussion.
- When asynchronous I/O is enabled, the kernel may attach a **wait queue** to a device-status table entry, allowing multiple outstanding requests.

2 Buffering

- **Double buffering** decouples a fast producer (e.g., modem) from a slower consumer (e.g., hard disk).
- While one buffer is being written to disk, the other is filled with new incoming data.

3 Caching

Definition: A **cache** is a fast memory region that holds copies of data residing elsewhere (typically on disk).

- The **buffer cache** is also used as a **page cache** for memory-mapped files, avoiding duplicate copies.
- Writes may be **delayed** (write-behind) to coalesce multiple small updates into a larger, more efficient disk write.

4 Spooling and Device Reservation

- **Spooling:** each printing job is written to a temporary file; a spooler daemon later feeds the printer sequentially.

- **Exclusive reservation**: a process can lock a device (e.g., via `open()` with exclusive flags) to prevent interleaved access.

5 Error Handling

- System calls return **-1** on failure and set **errno** to a specific error code (e.g., `ENODEV`, `EIO`).
- SCSI devices provide detailed sense data (key, additional sense code, qualifier) that can be retrieved by the OS for diagnostics.

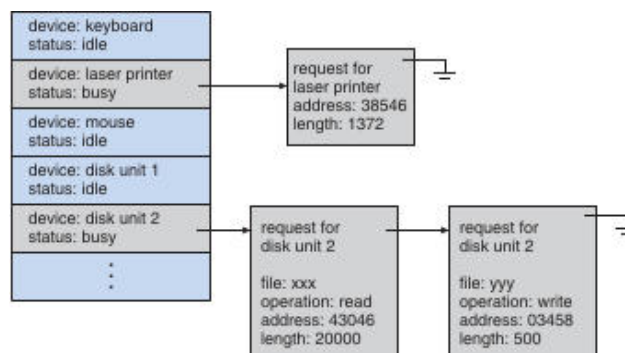
6 I/O Protection

- All I/O instructions are **privileged**; user processes must invoke a **system call** that executes in kernel mode.
- Memory-mapped I/O regions are protected by the MMU; only the kernel may map them into a process's address space, often after acquiring a lock (e.g., for graphics frame buffers).

7 Power Management

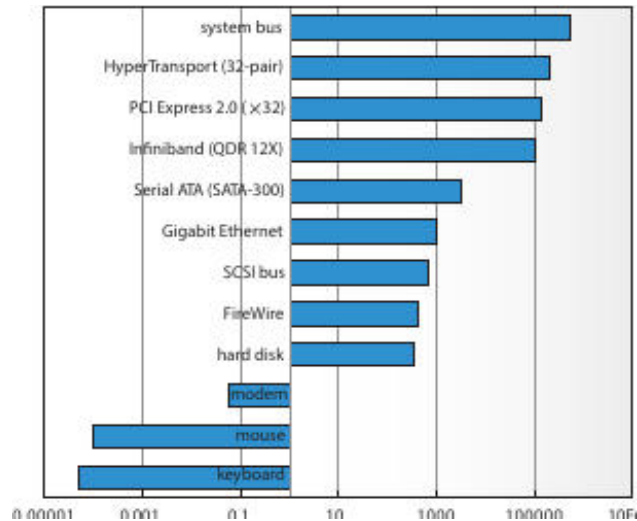
- Modern OSes support **deep sleep states** (power collapse) for individual components (CPU cores, I/O subsystems).
- When the system detects low load, it may **idle** devices, turn off clocks, or even shut down entire cores, dramatically reducing energy consumption—critical for both data-center efficiency and mobile battery life.

Device-status table example



The chart lists several devices (keyboard, printer, mouse, disks) together with their current status (idle/busy) and the specific I/O request associated with each. This visualizes how the kernel tracks pending operations and device availability.

Relative Speeds of Common Buses and Interfaces



The bar graph compares transfer rates from low-speed peripherals (mouse, modem) up to high-throughput links (PCI Express 2.0, InfiniBand). The wide range underscores why buffering and caching are essential to reconcile mismatched device speeds.

Unified I/O Model in the Kernel

- The kernel uses an **object-oriented dispatch table** (see the earlier UNIX open-file table) to route `read()`, `write()`, `ioctl()`, etc., to the appropriate driver routine, regardless of whether the underlying entity is a regular file, a raw disk, a network socket, or a character device.
- This abstraction allows **uniform system-call semantics** while letting each driver implement device-specific optimizations (e.g., sector-aligned reads for raw disks, packet framing for network interfaces).

These notes connect the concepts of linear and network I/O, timer services, blocking vs. asynchronous operations, vectored I/O, and the internal kernel mechanisms (scheduling, buffering, caching, spooling, protection, and power management) that together form a modern I/O subsystem.

Component-Level Power Management in Android ⚡

Device Tree & Usage Tracking

Component-level power management – infrastructure that understands the physical topology of a device and whether each component is currently in use.

- The **device tree** is a hierarchical representation of the hardware topology:
 - Flash storage** and **USB storage** are sub-nodes of the **I/O subsystem**.
 - The I/O subsystem is a sub-node of the **system bus**, which connects to the **CPU**.
- Each node is associated with a **driver** that tracks usage:
 - I/O pending to the flash (e.g., a file read).
 - Open references from an application (e.g., audio subsystem).
- Power-down decision flow**
 - If a component is idle → driver marks it as unused.

2. If *all* components under a bus are unused → the whole bus can be turned off.
3. If *all* nodes in the device tree are idle → the system may enter **power collapse** (deep sleep).

Wake Locks

Wake lock – a mechanism that lets an application temporarily prevent the kernel from entering power collapse.

- Applications **acquire** a wake lock before starting a long-running task (e.g., video playback, game rendering).
- While a wake lock is held, the kernel **blocks** power-collapse transitions.
- Upon completion, the application **releases** the lock, allowing the system to resume aggressive power saving.
- Typical use-case: the Android Market holds a wake lock while downloading and installing an update, ensuring the device stays awake until the process finishes.

I/O Kernel Subsystem Overview

Core Services (see §12.4.9)

Service	Role
Namespace management	Maps file and device names to internal objects.
Access control	Enforces permission checks for files/devices.
Operation control	Handles device-specific commands (e.g., modem dialing).
File-system allocation	Manages space for regular files and device files.
Device allocation	Tracks which drivers own which hardware resources.
Buffering / caching / spooling	Reduces latency and smoothes bursts.
I/O scheduling	Orders requests to minimize seek time and improve fairness.
Device-status monitoring	Detects errors, performs recovery, and reports failures.
Power management	Coordinates component-level power states (as described above).

Transforming Requests to Hardware

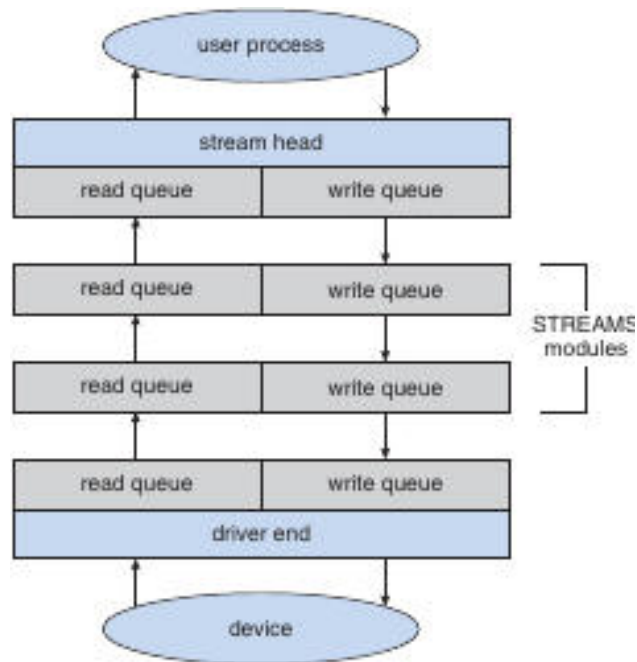
The I/O subsystem translates a high-level request (e.g., `read(fd, ...)`) into a series of low-level operations that reach the device controller.

UNIX style

- Devices appear as regular files within the **file-system namespace** (e.g., /dev/sda1).
- A **mount table** maps path prefixes to device names; the longest matching prefix determines the target device.
- The kernel resolves a path to a **major/minor device number**:
 - **Major number** → selects the driver.
 - **Minor number** → indexes the specific device instance within that driver.
- This unified namespace allows standard file-system permissions (ownership, mode bits) to apply to devices.

Aspect	MS-DOS	UNIX
Device delimiter	Colon (:))	Path (/dev/...)
Mapping method	Fixed table in kernel	Mount table + major/minor numbers
Permission model	Separate from file system	Integrated with file-system ACLs/permissions

Blocking Read Lifecycle (Detailed)



The figure expands the generic I/O flow to show the specific steps a blocking read() undergoes, including wait-queue handling and interrupt-driven completion.

Step-by-step:

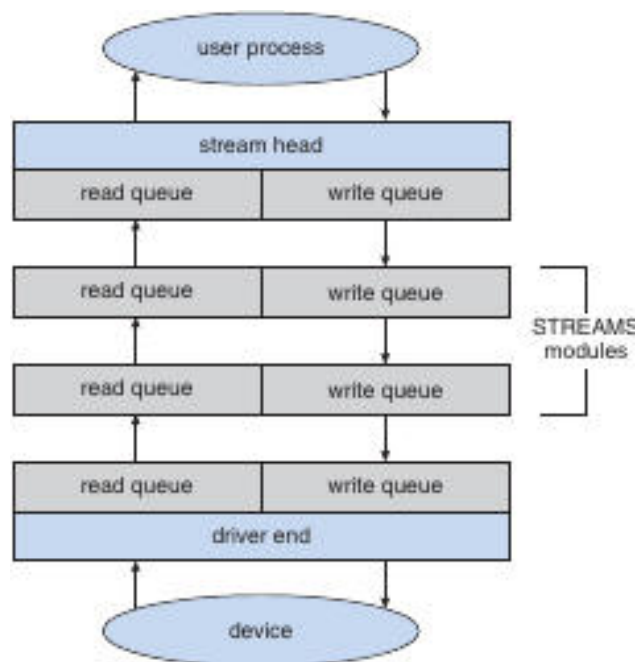
1. **Process issues read()** – system-call entry point.
2. **Parameter validation** – kernel checks buffer address and length.
3. **Cache hit?** – if data are already in the buffer cache, they are copied and the call returns.

4. **Cache miss** – the process is removed from the run queue and placed on the device's **wait queue**.
 5. **Driver request** – driver allocates a kernel buffer, programs the controller (often via DMA), and starts the transfer.
 6. **Interrupt** – controller signals completion; the interrupt handler stores any status, copies data to the kernel buffer, and wakes the waiting process.
 7. **Data copy** – kernel copies data from its buffer to the user buffer.
 8. **Process re-enqueue** – the scheduler places the process back on the ready queue.
-

STREAMS Architecture

STREAMS – a full-duplex framework that links a device driver to a user process through a series of modular queues.

- **Components**
 - **Stream head** – interfaces with the user process (handles `read()`, `write()`, `putmsg()`, `getmsg()`).
 - **Driver end** – connects to the physical device.
 - **Zero or more stream modules** – inserted dynamically via `ioctl()` to add processing (e.g., protocol handling, line editing).
- Each component maintains a **read queue** and a **write queue**; messages flow from one queue to the next.
- **Flow control**
 - *Flow-enabled* queues accept all messages and immediately forward them.
 - *Flow-disabled* queues buffer messages and signal upstream components to pause when full.



The diagram shows the user process at the top, the stream head, a chain of modules (each with read/write queues), and the device at the bottom. Messages traverse the queues in both directions.

- **I/O semantics**
 - `write()` copies raw data into the stream; `putmsg()` can attach control information.

- `read()` returns a byte stream; `getmsg()` returns a full message (data + control).
- The system is **asynchronous**: the process blocks only when the next queue cannot accept more data (flow-disabled).

- **Advantages**

- Modular driver development – the same module can be reused across multiple devices (e.g., a networking module used by Ethernet and Wi-Fi drivers).
- Message boundaries and control information are preserved, unlike simple byte-stream character devices.

Performance Considerations in I/O 🚀

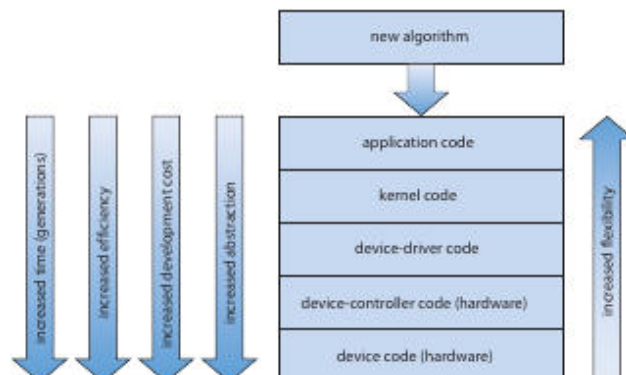
Sources of Overhead

Source	Description
Context switches	Transition between user process and kernel (system call) and back.
Interrupt handling	Saving/restoring CPU state, executing the handler, and signaling waiting processes.
Data copies	Between user buffers, kernel buffers, and device DMA buffers.
Polling vs. interrupts	Busy-waiting wastes CPU cycles; frequent interrupts increase handling cost.

Strategies to Reduce Overhead

1. **Minimize context switches** – batch I/O, use kernel-level threads for protocols (e.g., in-kernel telnet daemon).
2. **Reduce copies** – employ DMA, memory-mapped I/O, or zero-copy sockets.
3. **Interrupt moderation** – combine multiple events into a single interrupt, or use polling for high-throughput devices with predictable traffic.
4. **Offload work to hardware** – smart NICs that perform checksum or TCP segmentation offload.
5. **Balance subsystem capacities** – avoid a bottleneck in any of CPU, memory, bus, or I/O; otherwise other components idle.

Example: Impact of a New Algorithm Across Layers



The diagram shows a “new algorithm” influencing five layers: application code, kernel code, device-driver code, device-controller hardware, and the device itself, with arrows indicating increased efficiency and associated trade-offs (development cost, abstraction level, etc.).

- **Application level** – quickest to prototype, but incurs extra copies and context switches.
- **Kernel level** – eliminates user-kernel transitions, gains access to internal data structures (e.g., page cache).
- **Device-driver level** – can schedule DMA more efficiently, but requires careful synchronization.
- **Controller hardware** – lowest latency, but costly to redesign and harder to update.

Design Choices Across the I/O Stack

Layer	Typical Implementation	When to Prefer
Application	Prototype algorithms in user space; use existing syscalls.	Early experimentation, rapid iteration.
Kernel	Re-implement proven algorithms as kernel modules or built-in services.	Need for high performance, low latency, direct access to kernel resources.
Device driver	Optimize DMA descriptors, batch requests.	Device-specific tuning, reducing per-request overhead.
Device controller (hardware)	Embed complex logic (e.g., RAID parity calculation).	Very high throughput requirements, offloading CPU work.
Device (hardware)	Firmware updates for minor behavior changes.	When functionality can be expressed in microcode without hardware redesign.

- The **development cost** increases as we move down the stack, but **performance gains** typically improve.
- **Flexibility** decreases at lower levels; bugs become more critical (risk of system corruption).

Power Management Integration Recap

- **Component-level power management** uses the device tree and driver usage counters to decide when to shut down individual blocks (flash, USB, etc.).
- **Wake locks** give applications temporary authority to keep the system out of power collapse, ensuring long-running tasks complete.
- The combination of fine-grained power control and wake locks enables modern smartphones to aggressively reduce energy consumption while still supporting high-performance use cases.

Goals of Protection

Protection – a mechanism that controls the access of processes, users, or programs to system resources.

- Prevent **unauthorized use** of files, memory, CPU, and other resources.
- Detect **latent errors** early to avoid contamination of healthy subsystems.
- Enforce **policies** that may be fixed by design or defined by administrators and individual users.
- Provide **flexibility** so that different applications can impose varying access controls over time.

Principles of Protection

Principle of Least Privilege

Least-privilege – each program, user, or system component receives only the permissions required to perform its tasks.

- Analogy: a security guard with a passkey that opens only the public areas she guards.
- Limits damage from compromised components (e.g., a buffer overflow in a daemon should not allow execution of arbitrary code).
- Applied to **system calls**, **data structures**, and **features**; the OS should expose only the minimal set needed.

Need-to-Know

Need-to-know – a process may access a resource only when it currently requires it to complete its operation.

- Mirrors the least-privilege principle but emphasizes **temporal** restriction (access granted only for the duration of a specific phase).
- Example: a compiler invoked by a user should see only the source file and its generated output, not the user's unrelated private files.

Protection vs. Security

Aspect	Protection	Security
Focus	Controlling who can access what and how .	Ensuring integrity , confidentiality , and availability of data and resources.
Mechanism	Access-control lists, domains, capability lists, setuid bits.	Authentication, encryption, intrusion detection, audit trails.
Scope	Enforcement of policies at the OS level.	Broader assurance, including physical security and network defenses (covered in Chapter 14).

Protection mechanisms implement the *policies* that security relies on.

Domain-Based Protection

What Is a Domain?

Domain – a collection of access rights, each expressed as a pair .

- Example right: .
- A process executing in a domain can perform only the operations listed for each object.

Static vs. Dynamic Domains

Characteristic	Static Domains	Dynamic Domains
Association	Fixed for the lifetime of a process.	Can change during execution (e.g., phase-dependent access).
Implementation	Simple; no need to modify domain contents at runtime.	Requires mechanisms to switch domains or modify domain contents.
Flexibility	Limited; may grant excess rights to satisfy all phases.	Precise; rights can be tightened or expanded as needed.
Complexity	Low.	Higher – must manage safe domain transitions.

Domain Structure & Switching

- A domain is an **ordered set** of rights: .
- **Domain sharing**: multiple domains may contain the same right (e.g., shared by D2 and D3).
- **Domain switching** occurs when a process moves from one domain to another, either:
 1. **Explicitly** (system call, message passing).
 2. **Implicitly** (procedure call that carries its own domain).

Realizations of Domains

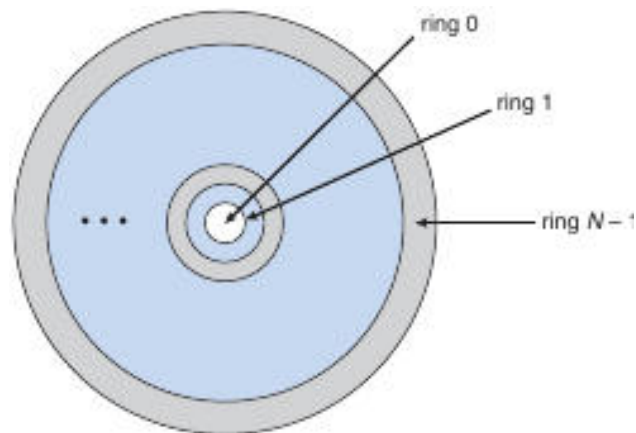
- **User-based domain** – the set of objects a user may access; switches on login/logout.
- **Process-based domain** – each process carries its own rights; switches when processes communicate.
- **Procedure-based domain** – a procedure's local variables define its domain; switches on procedure entry/exit.

UNIX Protection Model

- UNIX uses **user IDs (UIDs)** and the **setuid bit** on executable files to implement domain changes.
 - When a file's setuid bit is **on**, executing the file **temporarily** changes the process's UID to that of the file's owner.
 - When the setuid bit is **off**, the UID remains unchanged.
- Example flow:
 1. User A (UID A) runs a program owned by user B with setuid set.
 2. The process's UID becomes **B** for the duration of the program.
 3. Upon termination, the UID reverts to **A**.
- **Risks**: if a malicious user gains control of a setuid-root program, they obtain full system privileges.
- **Mitigations** (used in some UNIX variants):
 - Place privileged programs in a **special directory** whose execution automatically adopts root privileges, preventing hidden setuid binaries.
 - Use **daemon processes** that run with elevated rights and accept requests from ordinary users, avoiding setuid altogether.

MULTICS Ring Architecture

- MULTICS organizes protection domains as **hierarchical rings** (0 = most privileged).



The diagram shows concentric rings numbered from 0 (inner, highest privilege) to $N-1$ (outer, lowest privilege). A process's current ring number determines what segment it may access.

- Segment descriptors** contain:
 - Access bracket** (b_1, b_2) – allowed ring range for entry.
 - Limit** b_3 – maximum ring that may be entered from a lower-privilege ring.
 - Gate list** – entry points permitted for crossing rings.
- Ring transition rules:**
 - If a process in ring i calls a segment with bracket (b_1, b_2) :
 - Allowed when $b_1 \leq i \leq b_2$; the current ring remains unchanged.
 - If $i < b_1$, the call is allowed (moving to a *more privileged* ring); parameters referring to lower-privilege segments must be copied into the higher-privilege address space.
 - If $i > b_2$, the call is allowed only if $b_3 > i$ and the call targets a designated **gate**; otherwise the request is trapped.
- The ring model provides **fine-grained control** comparable to modern *ring-0/-3* CPU privilege levels, but with multiple intermediate rings for intermediate privileges.

Protection Mechanisms vs. Policies

Mechanism – **how** a protection decision is enforced (e.g., access-control list, capability token).

Policy – **what** is allowed or denied (e.g., “users may read files in /home but not /etc”).

- Separation of concerns** enables the same mechanism to support many policies without redesign.
- Example: an ACL mechanism can enforce both a *read-only* policy for a shared directory and a *write-only* policy for a log file simply by changing the ACL entries.

Access-Control Structures

Structure	Description	Typical Use
-----------	-------------	-------------

Access-Control List (ACL)	List of (principal, permission-set) pairs attached to an object.	File-system permissions, network resources.
Capability List	List of (object, rights) pairs owned by a subject (process or user).	Capability-based operating systems, fine-grained sandboxing.
Matrix Model	Two-dimensional table mapping subjects to objects with specific rights.	Theoretical foundation for many OS security models.
Setuid Bit	File-system flag that causes a process to adopt the file owner's UID on execution.	UNIX privilege escalation for specific programs.
Ring Numbers	Hierarchical privilege levels; a process's current ring restricts segment access.	MULTICS, modern CPU privilege rings.

Implementing Least-Privilege in Practice

- **Separate accounts** for each user, granting only the privileges needed for their role (e.g., a backup operator receives access solely to tape devices and backup directories).
- **Role-Based Access Control (RBAC)** – groups users into roles; each role has a predefined permission set.
- **Dynamic enabling/disabling** of privileged functions via system calls; audit trails record each privileged action for forensic analysis.

Summary of Key Concepts

- Protection is the **enforcement** layer that implements security policies.
- The **principle of least privilege** and **need-to-know** are central design goals, minimizing damage from faults or attacks.
- **Domains** encapsulate a set of rights; they can be **static** or **dynamic**, and may be associated with users, processes, or procedures.
- **UNIX** relies on UID/setuid mechanisms; **MULTICS** uses a multi-ring hierarchy with explicit gate control.
- Distinguishing **mechanisms** (how) from **policies** (what) yields flexible, adaptable protection architectures.